

Block

4

Implementation and Applications of AI

UNIT 10

Intelligent Agents

299

UNIT 11

Ai Programming with Python

322

UNIT 12

Applications of AI

372

BLOCK-4 INTRODUCTION

The Block-4, titled “**Implementation and Applications of AI**,” is comprised of three units. The details are as follows:

- **Unit 10: Intelligent Agents**
- **Unit 11: AI Programming with Python**
- **Unit 12: Applications of AI**

Unit 10 introduces intelligent agents as entities that perceive their environment through sensors and act upon it using actuators to achieve specific goals. It explains the concept of rational agents and rationality, along with the role of autonomy and learning in agent behavior. The unit also discusses different types of task environments using the PEAS framework and their characteristics, followed by the structure of agents and various agent models such as reflex, goal-based, utility-based, and learning agents. Finally, it highlights different forms of learning that enable agents to adapt and improve performance in dynamic environments.

Unit 11 introduces the fundamentals of Artificial Intelligence programming using Python, focusing on implementing AI concepts through practical coding. It covers essential Python libraries and tools used in AI, along with techniques for problem-solving, data handling, and algorithm development. The unit also emphasizes hands-on approaches to building intelligent applications, enabling learners to translate theoretical AI concepts into real-world programs.

The final unit of the block discusses the applications of AI in different areas: such as healthcare, finance, transportation, customer service and chatbots, education etc.

The final unit of the block highlights the diverse applications of Artificial Intelligence across various real-world domains such as healthcare, finance, transportation, education, and customer service. It discusses how AI techniques are used to solve practical problems, improve efficiency, and enhance decision-making in these fields. The unit also explores emerging applications like chatbots and intelligent systems, emphasizing the growing impact of AI technologies in everyday life.

UNIT 10 INTELLIGENT AGENTS

Structure	Page Nos.
10.0 Introduction	299
10.1 Objectives	300
10.2 Definitions	300
10.3 Agents and Rationality	302
10.3.1 Rationality vs. Omniscience	302
10.3.2 Autonomy and learning capability of the agent	303
10.3.3 Example: A boundary following robot	303
10.4 Task Environment of Agents	307
10.4.1 PEAS (Performance, Environment, Actuators, Sensors)	307
10.4.2 Example: An Automated Public Road Transport Driver	307
10.4.3 Different Types of Task Environments	308
10.4.3.1 Fully Observable vs. Partially Observable Environment	309
10.4.3.2 Static vs. Dynamic Environment	309
10.4.3.3 Deterministic vs. Stochastic Environment	309
10.4.3.4 Episodic vs. Sequential Environment	309
10.4.3.5 Single agent vs. Multi-agent Environment	310
10.4.3.6 Discrete Vs. Continuous Environment	310
10.4.4 Some Examples of Task Environments	311
10.4.4.1 Crossword Puzzle	311
10.4.4.2 Medical Diagnosis	311
10.4.4.3 Playing Tic-tac-toe	311
10.4.4.4 Playing Chess	312
10.4.4.5 Automobile Driver Agent	312
10.5 The Structure of Agents	312
10.5.1 SR (Simple Reflex) Agents	312
10.5.2 Model Based reflex Agents	313
10.5.3 Goal-based Agents	314
10.5.4 Utility-based Agents	316
10.5.5 Learning Agents	317
10.6 Different Forms of Learning in Agents	319
10.7 Summary	321
10.8 Solutions/Answers	321
10.9 Further Readings	321

10.0 INTRODUCTION

Since the time immemorial, we, the human beings, have always toyed with the idea of having some sort of slaves or agents, which would act as per our command, irrespective of the shape they take, as long as they do the job for which they have been designed or acquired. With the passage of time, human beings have developed different kinds of machines, where each machine has been intended to perform specific operation or a set of operations. However, **with the development of the computer, human aspirations have increased manifolds as it has allowed us to think of and actually implement non-human agents, which would show some level of independence and intelligence. Robot, one of the most popular non-human agents**, is a machine capable of perceiving the environment it is in, and further capable of taking some action or of performing some job either on its own or after taking some command.

Despite their perceived benefits, the dominant public image of the artificially embodied intelligent machines is more as potentially dangerous than potentially beneficial machines to the human race. The mankind is worried about the potentially dangerous capabilities of the robots to be designed and developed in the future.

Actually, any technology is a double-edged sword. For example, the Internet along with World Wide Web, on one hand, allows us to acquire information at the click of a button, but, at the same time, the Internet provides an environment in which a number of children (and, of course, the adults also) become addicts to downloading pornographic material. Similarly, **the development of non-human agents might not be without its tradeoffs. For example**, the more intelligent the robots are designed and developed, the more are the chances of a robot pursuing its own agenda than its master's, and more are the chances of a robot even attempting to destroy others to become more successful. Some intellectuals even think that, not in very distant future, there might be robots capable of enslaving the human beings, though designed and developed by the human beings themselves. Such concerns are not baseless. However, the (software) agents developed till today and the ones to be developed in the near future are expected to have very limited capabilities to match the kind of intelligence required for such behaviour.

In respect of the design and development of intelligent agents, with the passage of time, the momentum seems to have shifted from hardware to software, the latter being thought of as a major source of intelligence. But, obviously, some sort of hardware is essentially needed as a home to the intelligent agent.

10.1 OBJECTIVES

After going through this unit, you should be able to:

- define the concept of an agent;
 - explain the concepts of a 'rational agent' and 'rationality';
 - tell about the various task environments, for which the use of agents is appropriate to solve problems from;
 - explain the structure of an agent, and
 - discuss various forms of used for agents to learn.
-

10.2 DEFINITIONS

An **agent** may be thought of as an entity that acts, generally on behalf of someone else. More precisely, an **agent** is an entity that **perceives** its environment through **sensors** and **acts** on the environment through **actuators**. Some experts in the field require an agent to be additionally autonomous and goal directed also.

A **percept** may be thought of as an input to the agent through its sensors, over a unit of time, sufficient enough to make some sense from the input.

Percept sequence is a sequence of percepts, generally long enough to allow the agent to initiate some action.

In order to further have an idea about what a *computer agent* is, let us consider one of the first definitions of agent, which was coined by John McCarthy and his friends at MIT.

A software agent is a system which, when given a goal to be achieved, could carry out the details of the appropriate (computer) operations and further, in case it gets stuck, it can ask for advice and can receive it from humans, may even evaluate the appropriateness of the advice and then act suitably.

Essentially, a computer agent is a computer software that additionally has the following attributes:

- (i) it has autonomous control i.e., it operates under its own control
- (ii) it is perceptive, i.e., it is capable of perceiving its own environment
- (iii) it persists over a long period of time
- (iv) it is adaptive to changes in the environment and
- (v) it is capable of taking over others' goals.

As the concept of (software) agent is of relatively recent origin, different pioneers and other experts have been conceiving and using the term in different ways. There are two distinct but related approaches for defining an agent. The **first approach** treats an agent as an **ascription** i.e., the perception of a person (which includes expectations and points of view) whereas the **other approach** defines an agent on the basis of the **description** of the properties that the agent to be designed is expected to possess.

Let us first discuss the definition of agent according to first approach. Among the people who consider an agent as an ascription, a popular slogan is “**Agent is that agent does**”. In everyday context, **an agent is expected to act on behalf of someone** to carry out a particular task, which has been delegated to it. But to perform its task successfully, the agent must have knowledge about the domain in which it is operating and also about the properties of its current user in question. In the course of normal life, we hire different agents for different jobs based on the required expertise for each job. Similarly, a **non-human intelligent agent also is imbedded with required expertise of the domain as per requirements of the job under consideration**. For example, *a football-playing agent would be different from an email-managing agent*, although both will have the common attribute of modeling their user.

According to the second approach, an agent is defined as an entity, which functions continuously and autonomously, in a particular environment, which may have other agents also. **By continuity and autonomy of an agent, it is meant** that the agent must be able to carry out its job in a flexible and intelligent fashion and further is expected to adapt to the changes in its environment without requiring *constant* human guidance or intervention. Ideally, an agent that functions continuously in an environment over a long period of time would also **learn** from its experience. In addition, we expect an agent, which lives in a multi-agent environment, to be able to **communicate and cooperate** with them, and perhaps move from place to place in doing so.

According to the **second approach** to defining agent, **an agent is supposed to possess some or all of the following properties:**

- **Reactivity:** The ability of sensing the environment and then acting accordingly.
- **Autonomy:** The ability of moving towards its goal, changing its moves or strategy, if required, without much human intervention.
- **Communicating ability:** The ability to communicate with other agents and humans.
- **Ability to coexist by cooperating:** The ability to work in a multi-agent environment to achieve a common goal.
- **Ability to adapt to a new situation:** Ability to learn, change and adapt to the situations in the world around it.
- **Ability to draw inferences:** The ability to infer or conclude facts, which may be useful, but are not available directly.
- **Temporal continuity:** The ability to work over long periods of time.
- **Personality:** Ability to impersonate or simulate someone, on whose behalf the agent is acting.
- **Mobility:** Ability to move from one environment to another.

10.3 AGENTS AND RATIONALITY

Further, a **rational agent** is an agent that acts in a manner that achieves best outcome in an environment with certain outcomes. In an uncertain environment, a rational agent through its actions attempts the best-expected outcome.

It may be noted that *correct inferencing* is one of the several possible mechanisms for achieving *rationality*. However, sometimes a rational action is also possible without inferencing. For example, removing hand when a very hot utensil is touched *unintentionally* is an example of rationality based on *reflex action* instead of based on *inferencing*.

We discuss the concepts of **rationality** and **rational agent** in some more detail.

Attempting to take always the correct action, possibly but not necessarily involving logical reasoning, is only one part of being rational. Further, if a perfectly correct action or inference is not possible then taking an approximately correct, but, optimal action under the circumstances is a part of rationality.

Like other attributes, we need some *performance measure* (i.e., the criteria to judge the performance or success in respect of the task to be performed) to judge rationality. A good **performance measure for rationality** must be:

- Objective in nature
- It must be measurable or observable and
- It must be decided by the designer of the agent keeping in mind the problem or the set of problems for handling which the agent is designed.

In summary, rationality depends on:

- The performance measure chosen to judge the agent's activities.
- The agent's knowledge of the current environment or of the world in which it exists. The better the knowledge of the environment, the more will be probability of the agent taking an appropriate action.
- The length of the percept sequence of the agent. In the case of a longer percept sequence, the agent can take advantage of its earlier decisions or actions for similar kind of situations.
- The set of actions available to the agent.

From the previous discussion, we know that a **rational agent** should take an action, which **would correct its performance measure** on the basis of its knowledge of the world around it and the percept sequence.

By the *rationality of an agent*, we do not mean it to be always successful or it to be **omniscient**. **Rationality is concerned with** the agent's capabilities for **information gathering, exploration and learning** from its environment and experience and is also concerned with the **autonomy** of the agent.

10.3.1 Rationality vs. Omniscience

The basic difference between being rational and being omniscient is that **rationality** deals with trying to maximize the output on the basis of current input, environmental conditions, available actions and past experience whereas being **omniscient** means having knowledge of *everything*, including knowledge of the future i.e., what will be the output or outcome of its action. Obviously being omniscient is next to impossible.

In this context, let us consider the following scenario: Sohan is going to the nearby grocery shop, but unfortunately when Sohan is passing through the crossing suddenly a police party comes at that place chasing a terrorist and attempts to shoot the terrorist but unfortunately the bullet hits Sohan and he is injured.

Now the question is: *Is Sohan irrational in moving through that place.* The answer is no, because the human agent *Sohan* has no idea, nor is expected to have an idea, of what is going to happen at that place in the near future. Obviously, Sohan is *not omniscient* but, from this incident can *not be said to be irrational*.

10.3.2 Autonomy and Learning Capability of the Agent

Autonomy means the dependence of the agent on its own perceptions (what it perceives or receives from the environment through senses) rather than the prior knowledge of its designer. In other words, an agent is **autonomous** if it is capable of learning from its experience and has not to depend upon its prior knowledge which may either be not complete or be not correct or both. Greater the autonomy more flexible and more rational the agent is expected to be. So we can say that a rational agent should be autonomous because it should be able to learn to compensate for the incomplete or incorrect prior knowledge provided by the designer. In the initial stage of its operations, the agent may not, rather should not, have *complete autonomy*. This is desirable, in view of the fact that in the initial stage, the agent is yet to acquire knowledge of the environment should use the experience and knowledge of the designer. But, as the agent gains experience of its environment, its behavior should become more and more independent of its prior knowledge provided to it by its designer.

Learning means the agent can update its knowledge based on its experience and changes in the environment, for the purpose of taking better actions in future. Although the designer might feed some prior knowledge of the environment in the agent but, as mentioned earlier, as the environment might change with the passage of time, therefore, feeding complete knowledge in the agent, at the beginning of the operations is neither possible nor desirable. Obviously, there could be some extreme cases in which environment is static, i.e., does not change with time, but such cases are rare. In such rare cases, however, giving complete information about the environment may be desirable. So, in general, in order to update itself, the agent should have the learning capability to update its knowledge with the passage of time.

10.3.3 Example (A boundary following robot)

Let us consider the first **example of an agent**, which is *a robot that follows a boundary wall*.

The definition of the problem, to solve, which an agent is to be designed, consists in giving details of

- (i) The perception capabilities that are to be available to the agent
- (ii) The actions that the agent would be able to perform
- (iii) The environment in which the agent is expected to perform the task or solve the problem.

Environment and tasks

The robot is assumed to be enclosed in a room, which is perceived as a two-dimensional grid-space. The room is enclosed on all sides by walls high enough not to allow the robot to escape. The room may have some immovable objects. For the purpose of movement of the robot, the room is supposed to be divided into cells, in

the sense that at one time, the robot occupies only one cell and does not have its parts spread over two or more cells. To be capable of boundary-following behavior, the robot must be **able to sense** whether or not certain cells are free for it to possibly occupy in the next move. The **only action** it is able to take is **to move from the currently occupied cell to** any one of the unoccupied surrounding cells, e.g., to $c_1, c_2, \dots, c_8$ as shown in the following diagram.

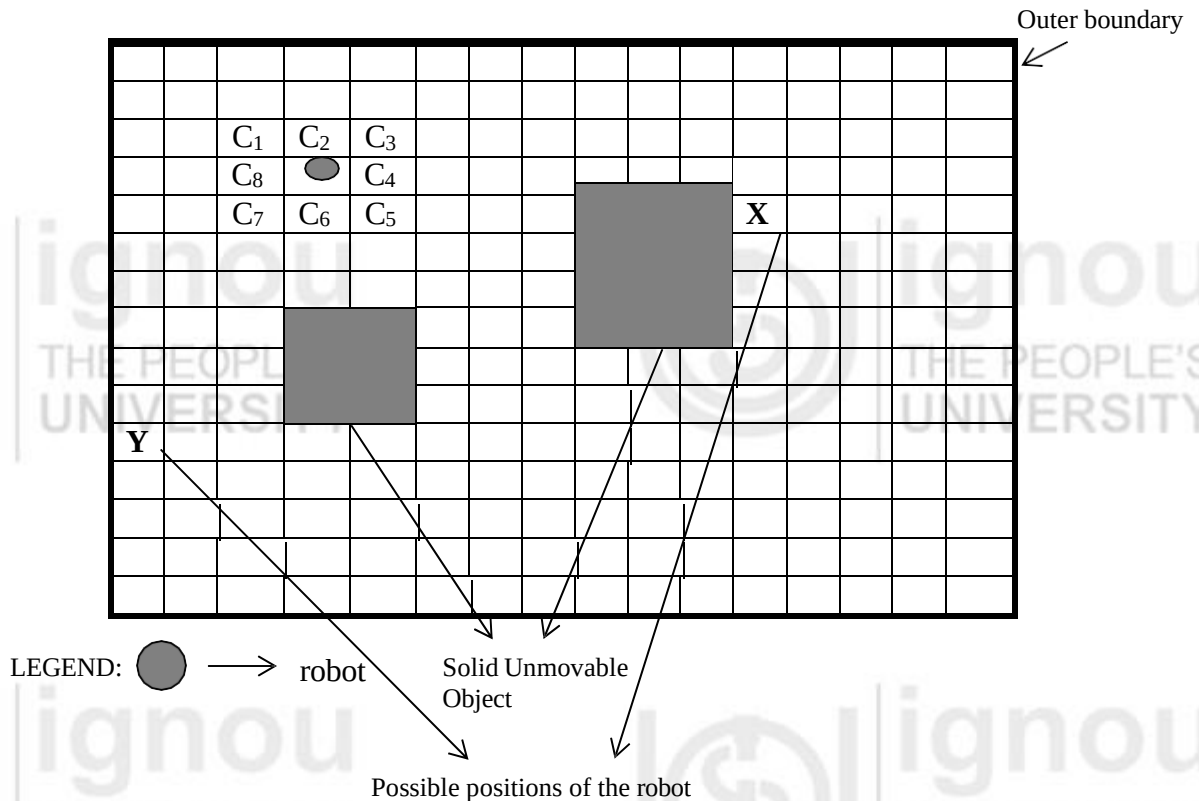


Figure 10.1: A Boundary Following Robot

The sensory input about the surrounding cells can be represented in the form of an tuple vector say $\langle c_1, c_2, c_3, c_4, c_5, c_6, c_7, c_8 \rangle$, where each c_i is a binary number i.e., c_i is either a 0 for representing a **free cell** or a 1 for representing the fact that the cell is **occupied** by some object. If, at some stage, the robot is in the cell represented by **X**, as shown in Figure 10.1, then the sense vector would be $\langle 0, 0, 0, 0, 0, 0, 0, 1 \rangle$. It may be noted that the corner and the boundary cells may not have exactly eight surrounding cells. However, for such cells, the missing neighbouring cell may be treated as occupied. For example, for the cell **Y** of Figure 10.1, the neighbouring cells to the left of the cell **Y** do not exist. However, we assume these cells exist but are occupied.

Thus, if the robot is in position **Y** of Figure 10.1 then the sense vector for the robot is $\langle 1, 0, 0, 0, 0, 0, 1, 1 \rangle$.

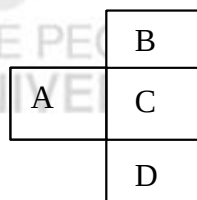
Also, the robot performs only one type of action i.e., it can move in one of the free cells adjacent to it. In case the robot attempts to move to a cell, which is not free, the action will have no effect i.e., there will be no action.

On the basis of the properties of the environment in which the robot is operating, **it is the job of the designer to develop an algorithm for the process that converts robot's precept sequences** (which in this case is the sensory input c_1 to c_8) **into corresponding actions**. Let us divide the task of determining the action the agent should take on the basis of the sensory inputs into two phases, viz., **perception** phase and **action** phase.

Perception Phase:

There is an enclosed space, which we call a room. The room is thought of as divided into cells. A cell has just enough space for the robot to be accommodated. Further, at any time, robot, except during transition, can not lie across two cells.

- Placement of the agent in some cell of the room is essentially random.
- Agent is to perform boundary following.
- The sensory input for the robot consists of the binary values $c_1, c_2, \dots, c_8$ in the eight cells adjacent to the position occupied by the robot. Because there are 8 adjacent cells, so the possible combinations of these values is 2^8 i.e., 256 combinations. The movement of the robot to any one of the unoccupied/free surrounding cells depends upon the robot's environmental conditions. For example, if the (traffic) signal to the right is green and some cells on the right of the cell currently occupied by the robots, are free then move to the uppermost cell of free cells to the right. For example,



if A is the cell currently occupied by the robot and cell B is occupied by some object, but cells C and D are free. And, further, signal on the right is green, then the robot moves to cell C. However, if both B and C are occupied, then the robot moves to cell D. These environmental conditions are determined in terms of what we call *features*. For the current problem, let us assume that there are 4 binary valued features of the agent, namely f_1, f_2, f_3 and f_4 and these features are used to calculate the action to be taken by the agent.

The rules for assigning binary values to features may be assumed to be as given below:

IF $c_2 = 1$ OR $c_3 = 0$ THEN $f_1 = 1$ ELSE $f_1 = 0$.

IF $c_4 = 0$ OR $c_5 = 1$ THEN $f_2 = 1$ ELSE $f_2 = 0$.

IF $c_6 = 1$ OR $c_7 = 1$ THEN $f_3 = 1$ ELSE $f_3 = 0$.

IF $c_8 = 1$ OR $c_1 = 1$ THEN $f_4 = 1$ ELSE $f_4 = 0$.

Action Phase:

After calculating the values of the feature set of the robot, a *pre-defined function* is used to determine boundary following action of the robot (i.e., the movement to one of the surrounding cell). In addition to specified actions determined by the features and the pre-defined function, there may be a default action of movement to a surrounding free cell. The default action may be executed when no action is possible.

In order to illustrate the type of possible actions, let us consider an example of specifying the rules for different actions to be performed by the robot:

Action 1: IF $f_1 = 0$ AND $f_2 = 0$ AND $f_3 = 0$ AND $f_4 = 0$
THEN move up to the right-most free surrounding cell. If no such cell is free then Attempt Action 3.

Action 2: IF $f_1 = 1$ AND $f_2 = 0$

THEN move topmost free surrounding cell on the right by one cell. If no such cell is free, then stop.

Action 3: IF $f_2 = 1$ AND $f_3 = 0$

THEN move down to the left-most free cell. If no such cell is free then attempt Action 2.

Action 4: IF $f_3 = 1$ AND $f_4 = 0$

THEN move to the bottom-most free cell on the left. If no such surrounding cell is free then move to action 1.

We can see that the combinations of the features values serve as conditions under which the robot would perform its actions.

Next, we enumerate below some of the possible parameters for performance measures in case of such a simple agent.

- The number of times agent has to reverse its earlier courses of actions (more times indicates lower performance).
- The maximum of the average of distances from the walls (the less the maximum, higher the performance).
- The boundary distance traversed in particular time period (the more the distance covered, better the performance).

In the next section, we discuss some of the commonly used terms in context of the Agents.

Exercise 1

On the basis of the figure given below, find the sense vector of the robot if:

- (a) It starts from the location L_1
- (b) It starts from the location L_2

The robot can sense whether the 8 cells adjacent to it are free for it to move into one of them. If the location of the robot is such that some of the surrounding cells do not exist, then these cells are, assumed to exist and further assumed to be occupied

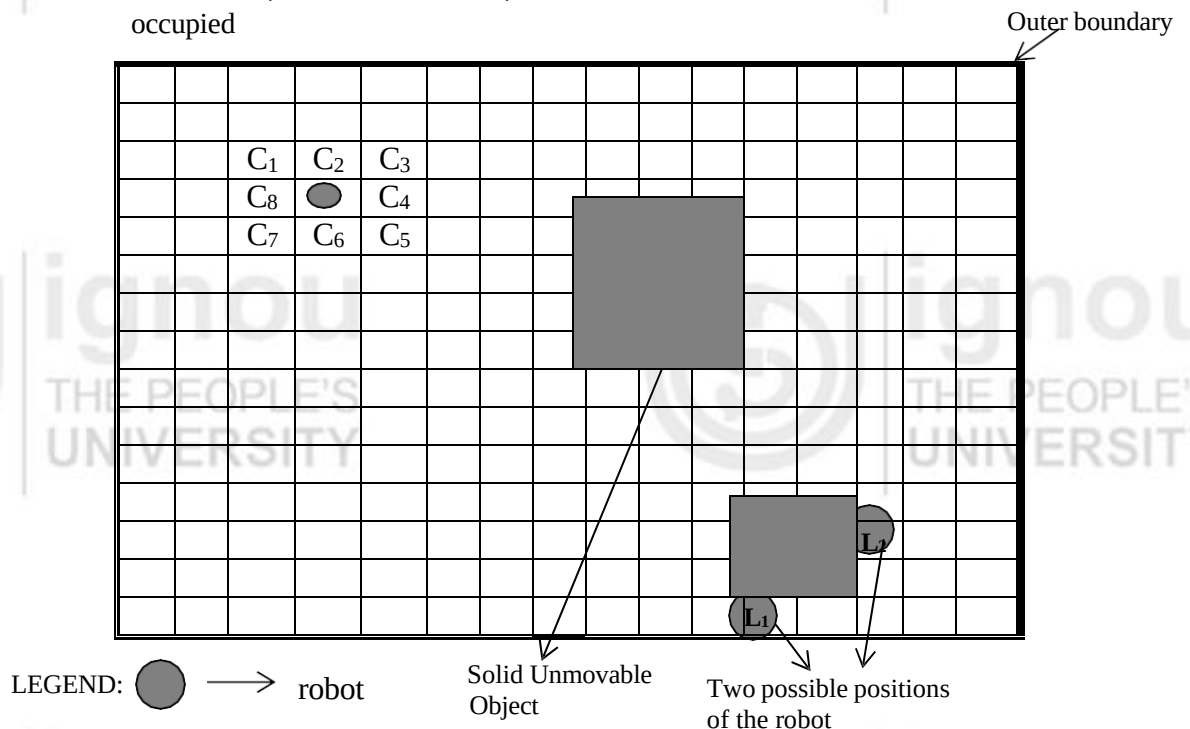


Figure 10.2 Environment of the robot

10.4 TASK ENVIRONMENT OF AGENTS

Task environments or problem environments are the environments, which include all the elements involved in the problems for which agents are thought of as solutions. Task environments will vary with every new task or problem for which an agent is being designed. Specifying the task environment is a long process which involves looking at different measures or parameters. Next we discuss a standard set of measures or parameters for *specifying* a task environment under the heading **PEAS**.

10.4.1 PEAS (Performance, Environment, Actuators, Sensors)

For designing an agent, the first requirement is to specify the task environment to the maximum extent possible. The task environment for an agent to solve one type of problems, may be described by the four major parameters namely, **performance** (which is actually the expected performance), **environment** (i.e., the world around the agent), **actuators** (which include entities through which the agent may perform actions) and **sensors** (which describes the different entities through which the agent will gather information about the environment).

The four parameters may be collectively called as PEAS.

We explain these parameters further, **through an example of an automated agent**, which we will preferably call **automated public road transport driver**. This is a much more complex agent than the simple **boundary following robot** which we have already discussed.

10.4.2 Example (An Automated Public Road Transport Driver Agent)

We describe the task environment of the agent on the basis of PEAS.

Performance Measures:

Some of the performance measures which can easily be perceived of an

automated public road transport driver would be :

- Maximizing safety of passengers
- Maximizing comfort of passengers
- Ability to reach correct destination
- Ability to minimize the time to reach the destination
- Obeying traffic rules
- Causing minimum discomfort or disturbance to other agents
- Minimizing costs, etc.

Environment (or the world around the agent)

We must remember that the environment or the world around the agent is extremely uncertain or open ended. There are unlimited combinations of possibilities of the environment situations, which such an agent could face. Let us enumerate some of the possibilities or circumstances which an agent might face:

- Variety of roads e.g., from 12-lane express-ways, freeways to dusty rural bumpy roads; different road rules including the ones requiring left-hand drive in some parts of the world and right-hand drive in other parts.
- The degree of knowledge of various places through which and to which driving is to be done.

- Various kinds of passengers, including high cultured to almost ruffians etc.
- All kind of other traffic possibly including heavy vehicles, ultra modern cars, three-wheelers and even bullock carts.

Actuators:

These include the following:

- Handling steering wheel, brakes, gears and accelerator
- Understanding the display screen
- A device or devices for all communication required

Sensors:

The agent acting as automated public road transport driver must have some way of sensing the world around it i.e., the traffic around it, the distance between the automobile and the automobiles ahead of it and its speed, the speeds of neighbouring vehicles, the condition of the road, any turn ahead etc. *It may use sensors like odometer, speedometer, sensors telling the different parameters of the engine, Global Positioning System (GPS)* to understand its current location and the path ahead. Also, there should be some sort of sensors to calculate its distance from other vehicles etc.

We must remember that the agent example *the automated public road transport driver*, which we have considered above, is quite difficult to implement. However, there are many other agents, which operate in comparatively simpler and less dynamic environments, e.g., a game playing robot, an assembly line robot control, and an image processing agent etc.

10.4.3 Different Types of Task Environments

Next, we discuss some of the important characteristics of the environments of the problems which are generally considered for solution through agent technology.

- Fully observable vs. partially observable
- Static vs. dynamic
- Deterministic vs. stochastic
- Episodic vs. sequential
- Single agent vs. multi-agent

10.4.3.1 Fully Observable vs. Partially Observable Environment

If the agent knows *everything* about its environment or the world in which it exists, through its sensors, then we say that the environment is **fully observable**. It would be quite convenient for the agent if the environment is a fully observable one because the agent will have a wholesome idea of what all is happening around before taking any action. The agent in a fully observable environment will have a complete idea of the world around it at all times and hence it need not maintain an internal state to remember and store what is going around it. **Fully observable environments are found rarely in reality.**

A **partially observable environment** is that in which the agent can sense **some** of the aspects or features of the world around it, but not all because of any number of reasons including the limitations of its sensors. Normally, there are greater chances of finding a partially observable environment in reality. *In the first example, viz., a boundary following robot*, which we have studied, the agent or robot has a limited

view of the environment around it, i.e., the agent has information only about the eight cells immediately adjacent to it in respect of whether each of these is occupied or free. It has no idea of how many non-moving objects are there within the room. Nor, it has an idea of its distance from the boundary wall. Also, *in the second example*, viz., **an automated public road transport driver**, the driver or the agent has a very restricted idea of the environment around it. It operates in a very dynamic environment, i.e., an environment that changes very frequently. Further, it has no idea of the number of vehicles and their positions on the road, the action the driver of the vehicle ahead is expected to take, the location of various potholes it is going to face in the next one kilometer and so on. So we can see that in both of these examples, the environment is partially observable.

10.4.3.2 Static vs. Dynamic Environment

Static environment means an environment which does not change while the agent is operating. Implementing an agent in a static environment is relatively very simple, as the agent does not have to keep track of the changes in the environment around it. So the time taken by an agent to perform its operations has no effect on the environment because the environment is static. The **task environment of the “boundary following robot” is static** because no changes are possible in the environment around it, irrespective of how long the robot might operate. Similarly, the environment of an agent solving a crossword puzzle is static.

A dynamic environment on the other hand may change with time on its own or because of the agent's actions. Implementing an agent in a dynamic environment is quite complex as the agent has to keep track of the changing environment around it. The **task environment of the “Automated public road transport driver” is an example of a dynamic environment** because the whole environment around the agent keeps on changing continuously.

In case an environment does not change by itself, but an agent's performance changes the environment, then the environment is said to be **semi-dynamic**.

10.4.3.3 Deterministic vs. Stochastic Environment

A deterministic environment means that the current state and the current set of actions performed by the agent will completely determine the next state of the agent's environment *otherwise*, the environment is said to be **stochastic**. The decision in respect of an environment being stochastic or deterministic, is taken from the point of view of the agent. If the environment is *simple and fully observable* then there are greater chances of its being *deterministic*. However, if the environment is *partially observable and complex* then it may be *stochastic* to the agent. For example, the *boundary following robot* exists in a deterministic environment whereas *an automated road transport driver agent* exists in a stochastic environment. In the later case, the agent has no prior knowledge and also it cannot predict the behavior of the traffic around it.

10.4.3.4 Episodic vs. Sequential Environment

In an episodic environment, the agent's experience is divided into *episodes*. An agent's actions during an episode depend only on that episode because subsequent episodes do not depend on previous episodes. For example, an agent checking the tickets of persons who want to enter the cinema hall, works in an episodic environment, as the process of checking the tickets of every new person is an episode which is independent of the previous episode i.e., checking the ticket of the previous person, and also the current action taken by the ticket checking agent does not effect the next action to be taken in this regard. Also, a robot while checking the seals on the

bottles on an assembly line, also works in an episodic environment as checking the seal of every new bottle is a new and independent episode.

In sequential environments there are no episodes and **the previous actions could affect the current decision or action and further, the current action could effect the future actions or decisions.** The task environment of “An automated public road transport driver” is an example of a sequential environment as any decision taken or action performed by the driver agent may have long consequences.

Comparatively, working in an episodic environment is much simpler for an agent than in a sequential environment, because, in the former case, previous actions do not effect the current actions of the agent. Also, while taking current action, the agent need not think of its future effects.

10.4.3.5 Single-agent vs. Multi-agent Environment

A single-agent environment is the simplest possible environment as the agent is the only active entity in the whole environment and it does not have to synchronize with the actions or activities of any other agent. Some of the examples of a single-agent environment are *boundary following robot, a crossword puzzle solving agent* etc.

If there is a possibility of other agents also existing in addition to the agent being described then the task environment is said to **be multi-agent environment.** In a multi-agent environment, the scenario becomes quite complex as the agent has to keep track of the behavior and the actions of the other agents also. There can be two general scenarios, one in which all the agents are working together to achieve some common goal, i.e., to perform a collective action. Such a type of environment is called **cooperative multi-agent environment.**

Also, it may happen that all or some of the agents existing in the environment are competing with each other to achieve something (for example, in a tic-tac-toe game or in a game of chess both of which are two agent games, each player tries to win by trying to predict the possible moves of the other agent), such an environment is called **competitive multi-agent environment.**

An **important issue in a multi-agent environment** is to determine whether other entities existing in the environment have to be treated as *passive objects or agents.* Normally, if an entity tries to optimize its performance which have an effect on the agent in question then that entity is treated as an agent, but there is no fixed rule to decide this fact. Also, **another very important issue** in a multi-agent environment is to decide how the **communication between different agents** will occur.

10.4.3.6 Discrete vs. Continuous Environment

The word discrete and continuous are related to the way time is treated, i.e., whether time is divided into slots (i.e., discrete quantities) or it is treated as a continuous quantity (continuous).

For example, the task environment of a “boundary following robot” or a “chess playing agent” is a discrete environment because there are finite number of discrete states in which an agent may find itself. On the other hand, *an automated public transport driver* agent’s environment is a continuous one, because, the actions of the agent itself as well as the environment around it are changing continuously. In the case of a driver agent, the values received through the sensor may be at discrete intervals but generally the values are received so frequently that practically the received values are treated as a stream of continuous data.

10.4.4 Some Examples of Task Environments

- (i) Planning crossword puzzle
- (ii) Medical diagnosis
- (iii) Playing tic-tac-toe
- (iv) Playing chess
- (v) Driving automobile

10.4.4.1 Crossword Puzzle

As we have already discussed the environment is **single agent** and **static** which makes it simple as there are no other players, i.e., agents and the environment does not change. But the environment is **sequential** in nature, as the current decision will affect all the future decisions also. On the simpler side, the environment of a crossword puzzle is **fully observable** as it does not require remembering of some facts or decisions taken earlier and also the environment is **deterministic** as the next state of the environment fully depends on the current state and the current action taken by the agent. Time can be divided into **discrete quanta** between moves and so we can say that the environment is discrete in nature.

10.4.4.2 Medical Diagnosis

The first problem in the task environment of an agent performing a medical diagnosis is to decide whether the environment is to be treated as a single-agent or multi-agent one. If the other entities like the patients or staff members also have to be viewed as agents (obviously based on whether they are trying to maximize some performance measure e.g., Profitability) then it will be a **multi-agent** environment **otherwise** it will be a **single** agent environment. To keep the discussion simple, in this case, let us choose the environment as **a single agent** one.

We can very well perceive that the task environment is **partially observable** as all the facts and information needed may not be available readily and hence, need to be remembered or retrieved. The environment is **stochastic** in nature, as the next state of the environment may not be fully determined only by the current state and current action. Diagnosing a disease is stochastic rather than deterministic.

Also the task environment is partially **episodic** in nature and partially **sequential**. The environment is episodic because, each patient is diagnosed, irrespective of the diagnoses of earlier patients. The environment is sequential, in view of the fact that for a particular patient, earlier treatment decisions are also taken into consideration in deciding further treatment. Furthermore, the environment may be treated as **continuous** as there seems to be no benefit of dividing the time in discrete slots.

10.4.4.3 Playing Tic-tac-toe

As it is a two-player game so obviously the environment is **multi-agent**. Moreover, both the players try to maximize their chances of winning, so it is a **competitive** one. Further, the environment is a **fully observable** environment. The environment here is **neither fully deterministic nor stochastic** as the environment is deterministic except for one fact that action of the other agent is unpredictable, so the environment is **strategic**. Obviously the environment is **semi-dynamic** as the environment itself does not change but the agents performance score does change with time. As time may be divided into discrete slots between moves so the environment may be viewed as **discrete**. Further, the current decisions while making a move affect all the future decisions also, therefore, the environment is **sequential** also.

10.4.4.4 Playing Chess

The environment of a chess-playing agent is also the same as is that of an agent-playing tic-tac-toe. It is also a two-player i.e., **multi-agent** game. **Some people treat the environment as fully observable** but actually for some of the rules require remembering the game history so the full environment is not observable from the current state of the chess board. Thus, **strictly speaking the environment is partially-observable**. Further, using the same arguments as given in case of tic-tac-toe, the environment is **strategic, semi-dynamic, discrete and sequential** in nature.

10.4.4.5 Automobile Driver Agent

We have already discussed one subclass of a general driver agent in detail, viz., *an automated public road transport driver* which may include a bus driver, taxi driver or auto driver etc. A driver agent might also be christened as a *cruise control agent*, which may include other types of transport like water transport or air transport also with some variations in their environments.

Coming back to *an automated public road transport driver*, we can see that this is one of the most complex environments discussed so far. So the environment is **multi-agent and partially observable** as it is not possible to see and assume what all other agents i.e., other drivers are doing or thinking. Also the environment is fully **dynamic**, as it is changing all the time as the location of the agent changes and also the locations of the other agents change. The environment is **stochastic** in nature as anything unpredictable can happen, which may not be perceived exactly as a result of the current state and the actions of various agents. The environment is **sequential** as any decision taken by the driver might affect or change the whole future course of actions. Also the time is **continuous** in nature although the sensors of the driver agent might work in discrete mode.

10.5 THE STRUCTURE OF AGENTS

There are two parts of an agent or its structure:

- A (hardware) device with sensors and actuators in which that agent will reside, called the *architecture* of the agent.
- An agent program that will convert or map the percepts in to actions.

Also, the agent program and its architecture are related in the sense that for a different agent architecture a different type of agent program is required and vice-versa. For example, in case of a *boundary following robot*, if the robot does not have the capability of sensing adjacent cells to the right, then the agent program for the robot has to be changed.

Next, we discuss different categories of agents, which are differentiated from each other on the basis of their agent programs. Capability to write efficient agent programs is the key to the success for developing efficient rational agents. Although the table driven approach (in which an agent acts on the basis of the set of all possible percepts by storing these percepts in tables) to design agents is possible yet the approach of developing equivalent agent programs is found much more efficient.

Next, we discuss some of the general categories of agents based on their agents programs:

- **SR (Simple Reflex) agents**
- **Model Based reflex agents**

- **Goal-based agents**
- **Utility based agents**

10.5.1 Simple Reflex (SR) Agents

These are the agents or machines that have no internal state (i.e., they don't remember anything) and simply react to the current percepts in their environments. An interesting set of agents can be built, the behaviour of the agents in which can be captured in the form of a simple set of functions of their sensory inputs. One of the earliest implemented agent of this category was called *Machina Speculatrix*. This was a device with wheels, motor, photo cells and vacuum tubes and was designed to move in the direction of light of less intensity and was designed to avoid the direction of the bright light. **A boundary following robot is also an SR agent.** For an automobile-driving agent also, some aspects of its behavior like applying brakes immediately on observing either the vehicle immediately ahead applying brakes or a human being coming just in front of the automobile suddenly, show the simple reflex capability of the agent. Such a simple reflex action in the agent program of the agent can be implemented with the help of simple condition-action rules.

For example: **IF** a human being comes in front of the automobile suddenly
THEN apply breaks immediately.

Although implementation of SR agents is simple, **yet on the negative side** this type of agents have very limited intelligence because **they do not store or remember anything**. As a consequence, they cannot make use of any previous experience. In summary, they do not learn. Also, **they are capable of operating correctly only if the environment is fully observable**.

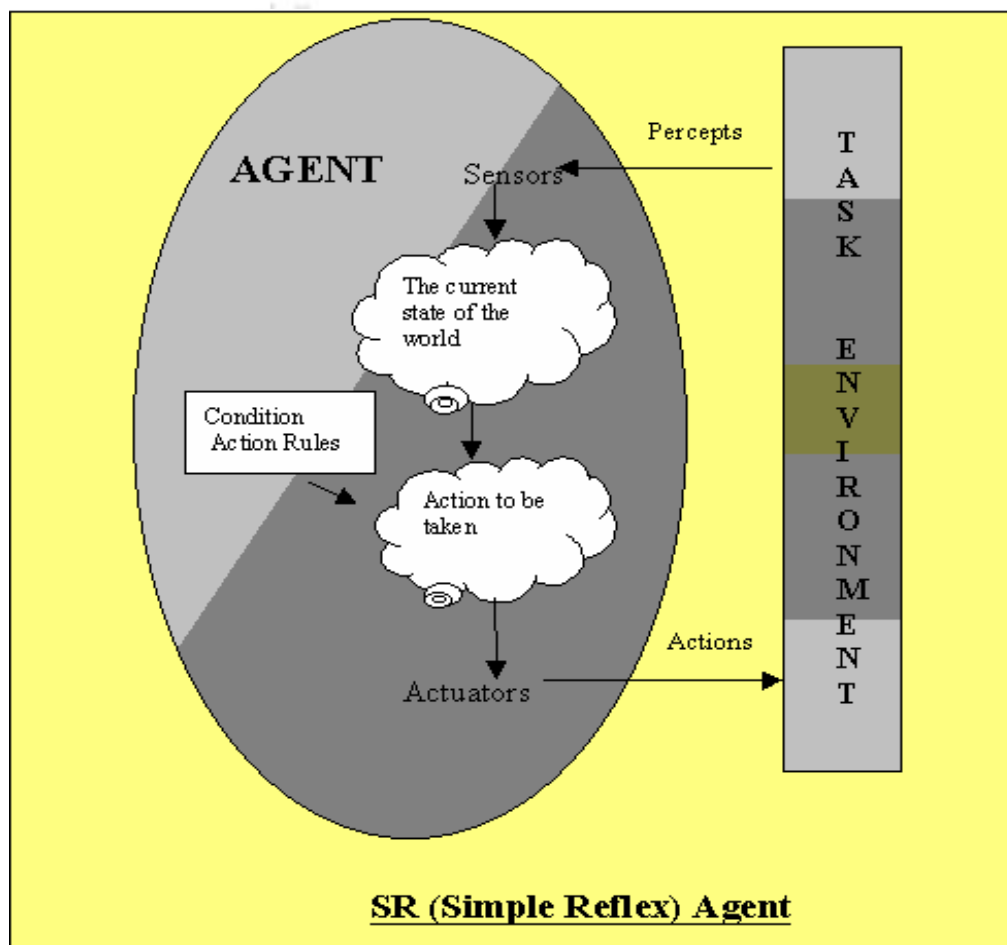


Figure 10.3: SR (Simple Reflex) Agent

10.5.2 Model-Based Reflex agents

Simple Reflex agents are not capable of handling task environments that are not fully observable. In order to handle such environments properly, in addition to reflex capabilities, the agent should maintain some sort of internal state in the form of a function of the sequence of percepts recovered up to the time of action by the agent. Using the percept sequence, the internal state is determined in such a manner that it reflects some of the aspects of the unobservable environment. Further, in order to reflect properly the unobserved environment, the agent is expected to have a model of the task environment encoded in the agent's program, where the model has the knowledge about–

- (i) the process by which the task environment evolves independent of the agent and
- (ii) effects of the actions of the agent have on the environment.

Thus, in order to handle properly the partial observability of the environment, the agent should have a model of the task environment in addition to reflex capabilities. Such agents are called **Model-based Reflex Agents**.

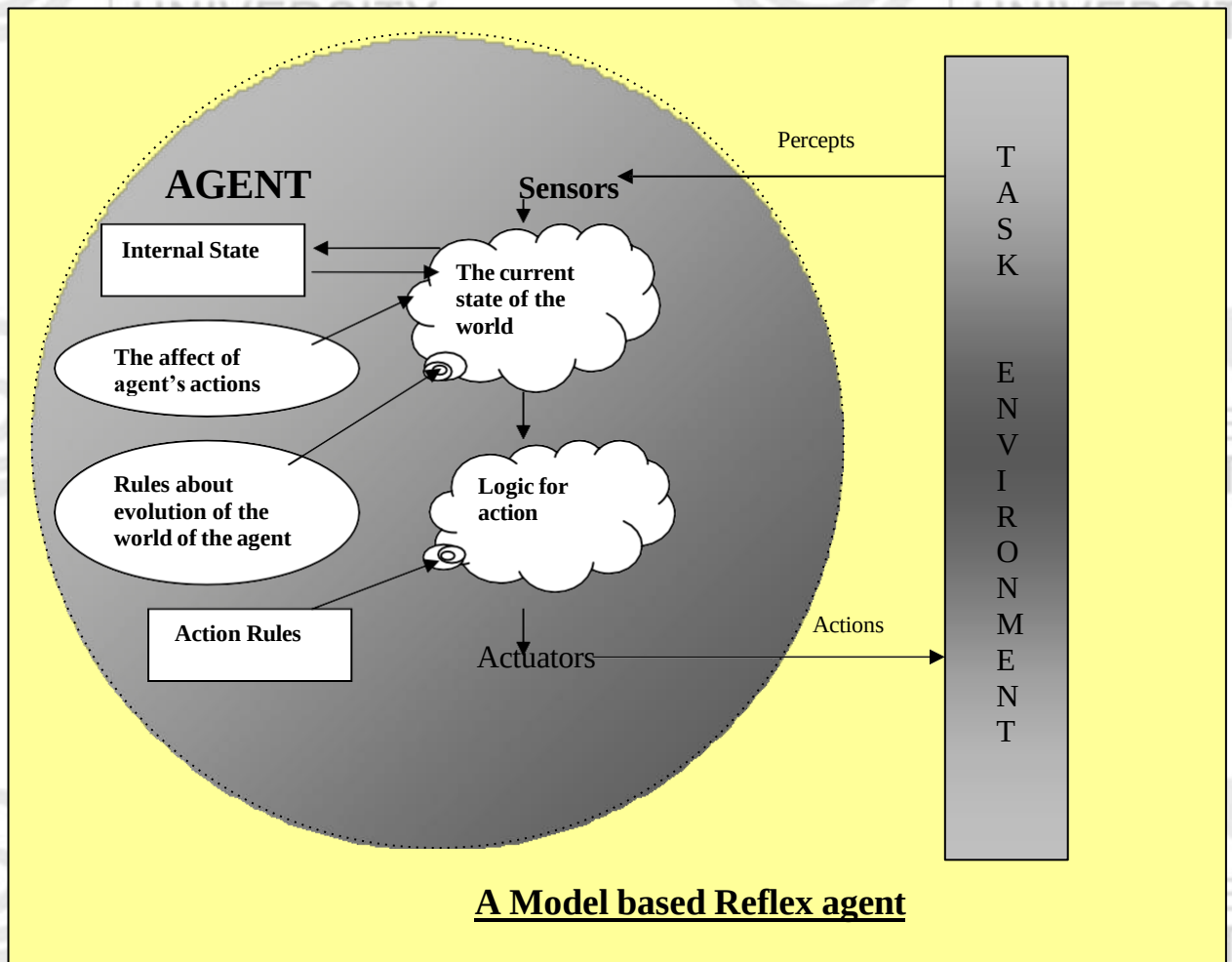


Figure 10.4: A Model based Reflex agent

10.5.3 Goal Based Agents

In order to design appropriate agent for a particular type of task, we know the nature of the task environment plays an important role. Also, it is desirable that the complexity of the agent should be minimum and just sufficient to handle the task in a particular environment. In this regard, first we discussed the simplest type of agents,

viz., Simple Reflex Agents. The action of this type of agent is decided by the current precept only. Next, we discussed the Model-Based Reflex Agents, for which an action is decided by taking into consideration not only the latest precept, but the whole precept history summarized in the form of internal state. Also, action for this type of agent is also decided by taking into consideration the knowledge of the task environment, represented by a model of the environment and encoded into the agent's program. However, in respect of a number of tasks, even this much knowledge may not be sufficient for appropriate action. For example, when we are going from city A to city B, in order to take appropriate action, it is not enough to know the summary of actions and path which has taken us to some city C between A and B. We also have to remember the goal of reaching to city B.

Goal based agents are **driven by the goal** they want to achieve, i.e., **their actions are based on the information regarding their goal, in addition to, of course, other information in the current state.** This goal information is also a part of the current state description and it describes everything that is desirable to achieve the goal. As mentioned earlier, an example of a goal-based agent is an agent that is required to find the path to reach a city. In such a case, if the agent is *an automobile driver agent*, and if the road is splitting ahead into two roads then the agent has to decide which way to go to achieve its goal of reaching its destination. Further, if there is a crossing ahead then the agent has to decide, whether to go straight, to go to the left or to go to the right. In order to achieve its goal, the agent needs some information regarding the goal which describes the desirable events and situations to reach the goal. The agent program would then use this goal information to decide the set of actions to take in order to reach its goal.

Another desirable capability which a good goal based agent should have is that if an agent finds that a part of the sequence of the previous steps has taken the agent away from its goal then it should be able to retract and start its actions from a point which may take the agent toward the goal.

In order to take appropriate action, decision-making process in goal-based agents may be simple or quite complex depending on the problem. Also, **the decision-making required by the agents of this kind needs some sort of looking into the future.** For example, it may analyze the possible outcome of a particular action before it actually performs that action. In other words, we can say that ***the agent would perform some sort of reasoning of if-then-else type***, e.g., an automobile driver agent having one of its goals as not to hit any vehicle in front of it, when finds the vehicle immediately ahead of it slowing down may not apply brakes with full force and in stead may apply brakes slowly so that the vehicles following it may not hit it.

As the goal-based agents may have to reason before they take an action, these agents might be slower than other types of agents but will be more flexible in taking actions as their decisions are based on the acquired knowledge which can be modified also. Hence, **as compared to SR agents** which may require rewriting of all the condition-action rules in case of change in the environment, the goal-based agents can adapt easily when there is any change in its goal.

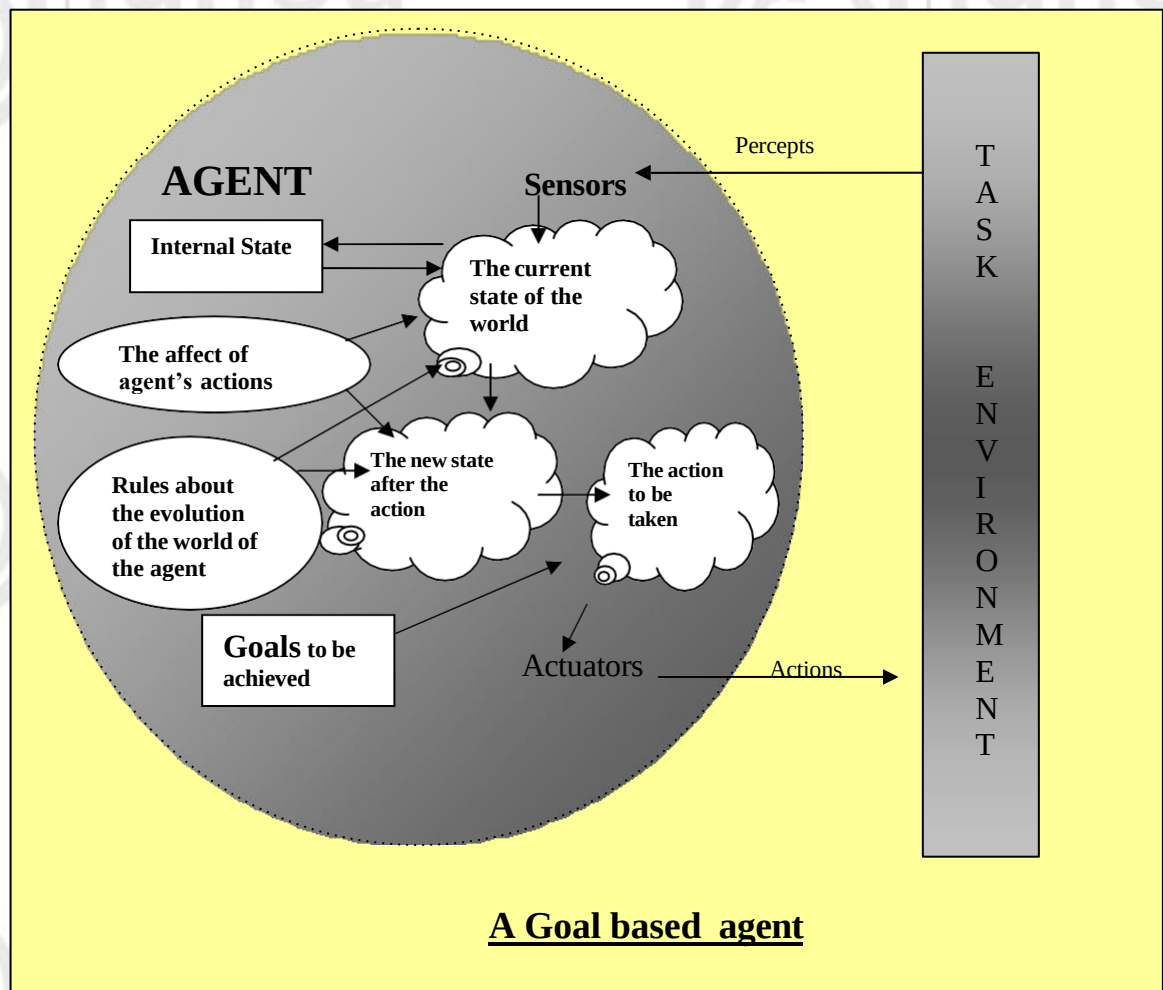


Figure 10.5: A Goal based agent

10.5.4 Utility Based Agents

Goal based agent's success or failure is judged in terms of its capability for achieving or not achieving its goal. A goal-based agent, for a given pair of environment state and possible input, only knows whether the pair will lead to the goal state or not. Such an agent will not be able to decide in which direction to proceed when there are more than one conflicting goals. Also, in a goal-based agent, there is no concept of partial success or somewhat satisfactory success. Further, if there are more than one methods of achieving a goal, then no mechanism is incorporated in a Goal-based agent of choosing or finding the method which is faster and more efficient one, out of the available ones, to reach its goal.

A more general way to judge the success or happiness of an agent may be, through assigning to each state a number as an approximate measure of its success in reaching the goal from the state. In case, the agent is embedded with such a capability of assigning such numbers to states, then it can choose, out of the reachable states in the next move, the state with the highest assigned number, out of the numbers assigned to various reachable states, indicating possibly the best chance of reaching the goal.

It will allow the goal to be achieved more efficiently. Such an agent will be more useful, i.e. will have more utility. A utility-based agent uses a **utility function**, which maps each of the world states of the agent to some degree of success. If it is possible

to define the utility function accurately, then the agent will be able to reach the goal quite efficiently. Also, a utility-based agent is *able to make decisions in case of conflicting goals*, generally choosing the goal with higher success rating or value. Further, in environments with multiple goals, the utility-based agent quite likely chooses the goal with least cost or higher utility goal out of multiple goals.

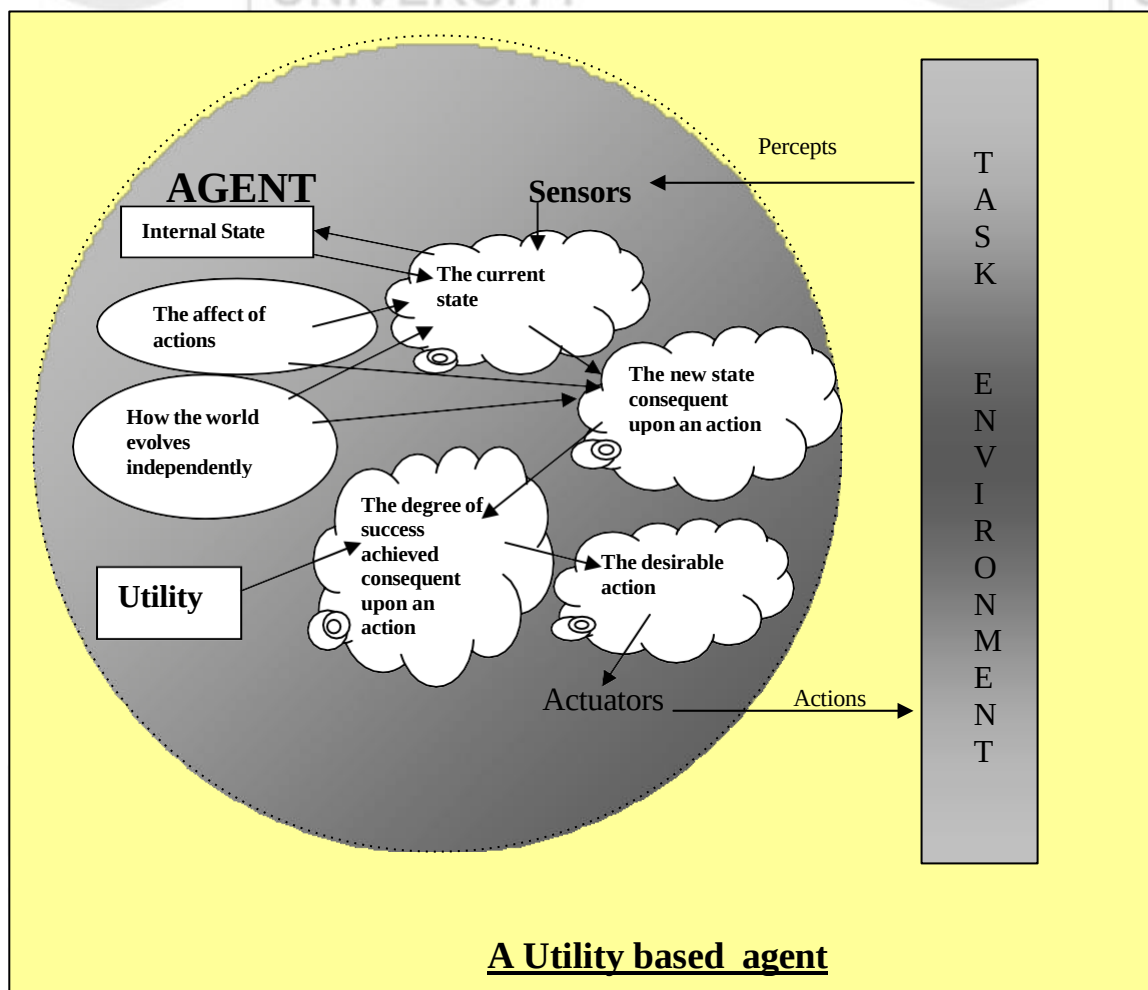


Figure 10.6: A Utility based agent

10.5.5 Learning Agents

It is not possible to encode all the knowledge in advance, required by a rational agent for optimal performance during its lifetime. This is specially true of the real life, and not just theoretical, environments. These environments are **dynamic** in the sense that the environmental conditions change, not only due to the actions of the agents under considerations, but due to other environmental factors also. For example, all of a sudden a pedestrian comes just in front of the moving vehicle, even when there is green signal for the vehicle. In a multi-agent environment, all the possible decisions and actions an agent is required to take, are generally unpredictable in view of the decisions taken and actions performed simultaneously by other agents. Hence, **the ability of an agent to succeed in an uncertain and unknown environment depends on its learning capability** i.e., its capability to change approximately its knowledge of the environment. For an agent with learning capability, some initial knowledge is coded in the agent program and after the agent starts operating, it learns from its actions the evolving environment, the actions of its competitors or adversaries etc. so as to improve its performance in ever-changing environment. If approximate learning component is incorporated in the agent, then the knowledge of the agent gradually

increases after each action starting from its initial knowledge which was manually coded into it at the start.

Conceptually the learning agent consists of four components:

- (i) **Learning Component:** It is the component of the agent, which on the basis of the percepts and the feedback from the environment, gradually improves the performance of the agent.
- (ii) **Performance Component:** It is the component from which all actions originate on the basis of external percepts and the knowledge provided by the learning component.

The design of learning component and the design of performance element are very much related to each other because a learning component is of no use unless the performance component can be designed to convert the newly acquired knowledge into better useful actions.

- (iii) **Critic Component:** This component finds out how well the agent is doing with respect to a certain fixed performance standard and it is also responsible for any future modifications in the performance component. ***The critic is necessary to judge the agent's success with respect to the chosen performance standard, specially in a dynamic environment.*** For example, in order to check whether a certain job is accomplished, the critic will not depend on external percepts only but it will also compare the current state to the state, which indicates the completion of that task.

- (iv) **Problem Generator Component:** This component is responsible for suggesting actions (some of which may not be optimal) in order to gain some fresh and innovative experiences. Thus, ***this component allows the agent to experiment a little*** by traversing sometimes uncharted territories by choosing some new and suboptimal actions. This may be useful, because the actions which may seem suboptimal in a short run, may turn out to be much better in the long run.

In the case of an *automobile driver agent*, this agent would be of little use if it does not have learning capability, as the environment in which it has to operate is totally dynamic and unpredictable in nature. **Once the automobile driver agent starts operating, it keeps on learning from its experiences, both positive and negative.** If faced with a totally new and previously unknown situation, e.g., encountering a vehicle coming from the opposite direction on a one-way road, the problem generator component of the driver agent might suggest some innovative action to tackle this new situation. Moreover, the learning becomes more difficult in the case of an automobile driver agent, because the environment is only partially observable.

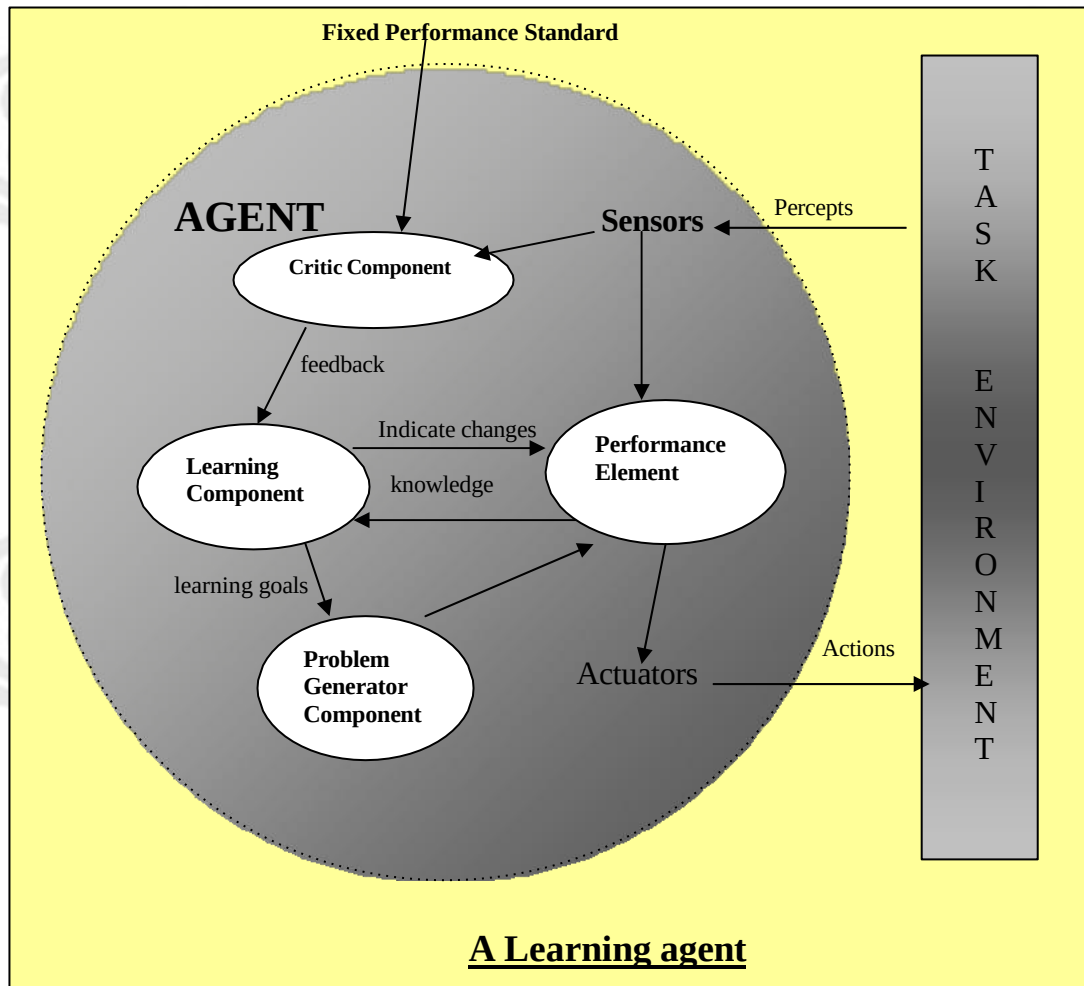


Figure 10.7: A Learning Agent

10.6 Different Forms of Learning in Agents

The purpose of embedding learning capability in an agent is that it should not depend totally on the knowledge initially encoded in it and on the external percepts for its actions. The agent learns by evaluating its own decisions and/or making observations of new situations it encounters in the ever-changing environment.

There may be various criteria for developing learning taxonomies. The criteria may be based on –

- The type of knowledge learnt, e.g., concepts, problem-solving or game playing,
- The type of representation used, e.g., predicate calculus, rules or frames,
- The area of application, e.g., medical diagnosis, scheduling or prediction.

We mention below another classification, which is independent of the representation method and knowledge representation used. Under this classification scheme, there are five learning methods:

- (i) Rote learning or memorization
- (ii) Learning through instruction
- (iii) Learning by analogy
- (iv) Learning by induction and
- (v) Learning by deduction.

Rote learning is the simplest form of learning, which involves least amount of inferencing. In this form of learning, the knowledge is simply copied in the knowledge base. This is the learning, which is involved in memorizing multiplication tables.

Next type of learning is *learning through instruction*. This type of learning involves more inferencing because the knowledge in order to be operational should be integrated in the existing knowledge base. This is the type of learning that is involved in the learning of a pupil from a teacher.

Learning by analogy involves development of new concepts through already known similar concepts. This type of learning is commonly adopted in textbooks, where some example problems are solved in the text and then exercises based on these solved examples are given to students to solve. Also, this type of learning is involved when, on the basis of experience of driving light vehicles, one attempts to drive heavy vehicles.

Learning by induction is the most frequently used form of learning employed by human being. This is a form of learning which involves inductive reasoning — a form of reasoning under which a conclusion is drawn on the basis of a large number of positive examples. For example, after seeing a large number of cows, we conclude a cow has four legs, white colour, two horns symmetrically placed on the head etc. Inductive reasoning, though usually leads to correct conclusions, yet the conclusions may not be *irrefutable*.

For instance, in the case of the concept of cow as discussed above, we may find a black cow, or we may find a three-legged cow who has lost one leg in an accident or a single-horn cow.

Finally, we discuss *deductive learning*, which is based on deductive inference — an *irrefutable* form of reasoning. By *irrefutable* form of reasoning we mean that the conclusion arrived at through deductive (i.e., any *irrefutable*) reasoning process is always correct, if the hypotheses (or given facts) are correct. This is the form of reasoning which is dominantly applied in Mathematics.

Inductive learning occupies an important place in designing the learning component of an agent. The learning in an agent is based on—

- The subject matter of learning, e.g., concepts, problem-solving or game playing etc.
- The representation used, predicate calculus, frame or script etc.
- The critic, which gives the feedback about the overall health of the agent.

Learning based on feedback is normally categorized as:

- Supervised learning
- Unsupervised learning
- Reinforcement Learning.

Supervised Learning: It involves a learning function from the given examples of its inputs and outputs. Some of the examples of this type of learning are:

- Generating useful properties of the world around from the percepts
- Mapping from conditions regarding the current state to actions
- Gathering information about the way world evolves.

Unsupervised Learning: In this type of learning, the pair of inputs and corresponding expected outputs are not available. Hence, the learning system, on its own, has to find

relevant properties from the otherwise unknown objects. For example, finding shortest path, without any prior knowledge, between two cities in a totally unknown country.

Reinforcement (Rewards) Learning: *In many problems, the task or the problem is only realized, but cannot be stated.* Further, the task may be an ongoing one. The user expresses his or her satisfaction or dissatisfaction regarding the performance of the agent by occasionally giving the agent positive or negative rewards (i.e., reinforcements). ***The job of the agent is to maximize the amount of reward (i.e., reinforcement) it receives.*** In case of a simple goal achieving problem, the agent can be rewarded positively when it achieves the goal and punished every time it fails to do so.

In this kind of task environment, an action policy is needed to maximize reward. But *sometimes in case of ongoing and non-terminating tasks the future reward might be infinite, so it is difficult to decide how to maximize it.* In such a scenario, a method of progressing ahead is to discount future rewards beyond a certain factor. i.e., the agent may prefer rewards in the immediate future to those in the distant future.

Learning action policies in environments in which rewards depend on a sequence of earlier actions is called **delayed-reinforcement learning**.

10.7 SUMMARY

In this unit, the concept of 'Intelligent Agent' is introduced and further various issues about intelligent agents are discussed. Some definitions in this context are given in section 10.2. The next section discusses the concept of rationality in context of agents. Section 10.4 discusses various types of task environments for agents. Next section discusses structure of an agent. Finally, section 10.6 discusses various forms of learning in an agent.

10.8 SOLUTIONS/ANSWERS

Ex. 1) For L_1

The left upper cell is unoccupied, therefore, $C_1=0$, $C_2=0=C_3=C_4$ and $C_8=0$
 However, as C_5 , C_6 and C_7 do not exist hence are assumed to be occupied.
 Thus, the sense-vector is (0, 0, 0, 0, 1, 1, 1, 0)

For L_2

$C_1=1=C_8$ and $C_2=C_3=C_4=C_5=C_6=C_7=0$
 Thus, the sense-vector is (1, 0, 0, 0, 0, 0, 0, 1)

10.9 FURTHER READING

1. Russell S. & Norvig P, *Artificial Intelligence: A Modern Approach* (Second Edition), (Pearson Education, 2003).

UNIT 11: AI PROGRAMMING WITH PYTHON

Structure	Page No.
11.1 Introduction to Python for AI	322
11.2 Objectives	323
11.3 Python Libraries for AI	323
11.4 Hands-on Implementations of:	349
11.4.1 N-Queen Problem	349
11.4.2 Min-Max Algorithm and Alpha-Beta Pruning	351
11.4.3 Breadth First Search	357
11.4.4 Depth First Search	359
11.4.5 Best First Search	360
11.4.6 A* Algorithm	361
11.4.7 AO* Algorithm	364
11.4.8 Iterative Deepening A*	366
11.5 Model Evaluation and Hyperparameter Tuning in Python	368
11.6 Summary	370
11.7 Solutions/Answers	370
11.8 Further Readings	371

11.1 INTRODUCTION TO PYTHON FOR AI

Python is a high-level, beginner-friendly, general-purpose programming language known for its readable syntax and wide range of applications. It has emerged as an important programming language for Artificial Intelligence (AI) and Machine Learning (ML).

Python has been adopted by most of the researchers, developers, and organizations to build AI-based applications because of the following reasons:

1. Python has easy syntax and human-readable code, which makes it accessible for both beginners and experienced programmers. AI development often involves complex algorithms and mathematical computations, and Python's ease of writing and debugging reduces development time and effort.
2. Python has a collection of several libraries that simplify AI and ML development. Some of the most widely used libraries include NLTK, OpenCV, Pandas, etc.
3. Python has a massive global community that actively contributes to open-source projects, which ensures continuous improvements and support. The availability of the extensive documentation, forums, and online courses makes learning and troubleshooting easy.
4. Python is a platform-independent language, which means that the applications developed in Python can run on various operating systems without any modification. This makes it an excellent choice for deploying AI solutions across different environments.

AI has multiple approaches to make the machines intelligent. Earlier, AI algorithms were based on rule-based systems and search techniques which were using logical reasoning and problem-solving methods. Later, the field has

shifted towards machine learning, where computers can learn the patterns from the data instead of relying solely on pre-defined rules.

Since Python has a rich set of libraries, it supports all these approaches. From building expert systems using rules to search-based problem-solving, and to modern deep learning models, Python provides a unified platform that can connect the history and future of AI. Some of the applications of Python in AI are:

- *Pathfinding in Robotics and Games* - Robots use state space search algorithms: A* or BFS/DFS to navigate in an environment.
- *Puzzle Solving* - Classic AI problems like the 8-puzzle, Sudoku, or N-Queens are solved using state-space search in Python.
- *Planning in Education Systems* - Intelligent tutoring systems search the space of teaching actions to lead students from current knowledge to mastery.
- *Machine Learning and Deep Learning* - Python enables the development of intelligent systems that can learn from data, recognize patterns, and make predictions. It is used in analytics, recommendation systems, and fraud detection.
- *Natural Language Processing (NLP)* - With libraries like NLTK, spaCy and transformers, Python is extensively used in NLP tasks such as sentiment analysis, chatbot development and automated translation.
- *Computer Vision* - Python's OpenCV library and deep learning frameworks help in image processing, facial recognition and object detection, making it essential in security, healthcare and autonomous driving applications.
- *Robotics and Automation* - Python plays a crucial role in robotic process automation (RPA) and autonomous systems, allowing machines to perform tasks with minimal human intervention.
- *Reinforcement Learning* - Python is used in AI-driven gaming, robotics and decision-making systems through reinforcement learning algorithms.

11.2 OBJECTIVES

After going through this unit, you should be able to:

- Understand how to use Python for AI-based applications
- Use available Python libraries
- Implement AI-based algorithms in Python

11.3 PYTHON LIBRARIES FOR AI

Python has a rich set of libraries to simplify various AI-based complex tasks. Python library such as Numpy and Pandas is used to process large amount of numerical data. Matplotlib is the library used for visualization of results. In rule-based AI systems, Python offers libraries such as Pyke and Experta, which help developers create knowledge bases and inference engines for decision-making. These libraries make it easy to represent rules, perform logical reasoning, and

build expert systems without starting from scratch. For state-space search, Python provides efficient implementations of graph and search algorithms through libraries like NetworkX, enabling exploration of paths, optimization problems, and heuristic-based approaches such as A*, IDA*, or even game-tree search using minimax.

Further, scikit-learn is one of the most widely used libraries for machine learning (ML). It offers a number of algorithms for classification, regression, clustering, dimensionality reduction and reinforcement learning. For deep learning (DL), Python provides support for the frameworks like TensorFlow and PyTorch which can be used to build neural networks, reinforcement learning systems and large-scale AI applications. Natural Language Processing (NLP) is also supported by libraries such as NLTK, spaCy and transformers (Hugging Face), which provide tools for sentiment analysis, chatbot development and machine translation. In the field of computer vision, OpenCV along with deep learning frameworks can be used for the tasks such as image recognition, medical imaging analysis and autonomous vehicle perception. Additionally, Python supports reinforcement learning using libraries like Stable Baselines3 and Ray RLLib which simplifies building agents that learn through trial and error in simulated environments. Following section will discuss some of these libraries in detail.

11.3.1 NumPy

AI models work with large amounts of numerical data and handling this efficiently is a key challenge. In Python, NumPy (short for Numerical Python) is a library that provides tools to work with arrays, matrices and numerical operations. It is the foundation of many other scientific and machine learning libraries like Pandas, SciPy, TensorFlow and PyTorch. It provides efficient data structures, mathematical functions, random number generation and linear algebra tools. It can be used for preprocessing data, performing matrix operations, or implementing simple AI models.

If you are using Anaconda, NumPy is already installed. Otherwise, you can install it using:

```
pip install numpy
```

To import NumPy library:

```
import numpy as np
```

Here, np is just an alias to make typing shorter. You can assign any name to create the alias.

For AI, we often work with **2D arrays** (matrices) and **3D arrays** (image data, sequences, etc.). This can be handled using NumPy arrays. This is similar to Python lists but is more powerful. You can create NumPy arrays in several ways:

```
#CODE: Arrays in python
import numpy as np

# From a Python list
arr = np.array([1, 2, 3, 4])
print("Python list is: ",arr)

# Multi-dimensional array
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
print("2-D array is ",arr2d)

# Using built-in functions
zeros = np.zeros((3,3))      # 3x3 matrix of zeros
ones = np.ones((2,2))       # 2x2 matrix of ones
randoms = np.random.rand(3,3) # 3x3 matrix with
random numbers
print("3x3 matrix of zeros", zeros)
print("2x2 matrix of ones",ones)
print("3x3 matrix with random numbers", randomnesss)
```

OUTPUT:

```
Python list is:
 [1 2 3 4]

2-D array is
 [[1 2 3]
 [4 5 6]]

3x3 matrix of zeros
 [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

2x2 matrix of ones
 [[1. 1.]
 [1. 1.]]

3x3 matrix with random numbers
 [[0.0368857  0.65398728 0.80446459]
 [0.20939835 0.79966979 0.92998546]
 [0.69515611 0.52541098 0.21407825]]
```

Every array has some properties that are useful such as shape, dimensions, data types, size, etc. These properties can be seen as follows:

```
#CODE: Arrays properties in python

a2d = np.array([[1, 2, 3], [4, 5, 6]]) #initialize
a 2-D array
```

```

print("Shape (rows, columns): ", a2d.shape) # Shape
(rows, columns)
print("Number of dimensions: ", a2d.ndim) # Number
of dimensions
print("Data type: ", arr2d.dtype) # Data type
print("Total number of elements: ", a2d.size) #
Total number of elements

```

OUTPUT:

```

Shape (rows, columns): (2, 3)
Number of dimensions: 2
Data type: int64
Total number of elements: 6

```

Once array is created, next step is to access its element either one by one (using indexing) or a subset of elements together (using slicing).

```

#CODE: Arrays Indexing and Slicing

a = np.array([10, 20, 30, 40, 50])
print("First element:", a[0]) # First element
print("Last element:", a[-1]) # Last element

b = np.array([[1, 2, 3], [4, 5, 6]])
print("Row 0, Column 1:", b[0, 1]) # Row 0, Column
1 -> 2

c = np.array([10, 20, 30, 40, 50])
print("Elements from index 1 to 3:", c[1:4]) #
Elements from index 1 to 3
print("First three elements:", c[:3]) # First
three elements
print("From index 2 to end:", c[2:]) # From index
2 to end

d = np.array([[1,2,3], [4,5,6], [7,8,9]])
print("First two rows, last two columns:", d[0:2,
1:3]) # First two rows, last two columns

```

OUTPUT:

```

First element: 10

Last element: 50

Row 0, Column 1: 2

Elements from index 1 to 3: [20 30 40]

First three elements: [10 20 30]

From index 2 to end: [30 40 50]

```

```
First two rows, last two columns:  
[[2 3]  
 [5 6]]
```

Further, NumPy arrays allow vectorized operations, meaning we can perform arithmetic directly on arrays without loops. This ability is very important in AI, where datasets contain thousands of numbers and we need to process them quickly.

```
# CODE: Array Operations  
  
x = np.array([1, 2, 3])  
y = np.array([4, 5, 6])  
  
print("Addition of two Vectors:", x + y)  
print("Subtraction of two Vectors:", x - y)  
print("Multiplication of two Vectors:", x * y)  
print("Division of two Vectors:", x / y)  
  
print("Scalar multiplication of each element by 2:",  
x * 2)  
print("Taking square of each element:", x ** 2)
```

OUTPUT:

```
Addition of two Vectors: [5 7 9]  
Subtraction of two Vectors: [-3 -3 -3]  
Multiplication of two Vectors: [ 4 10 18]  
Division of two Vectors: [0.25 0.4  0.5 ]  
Scalar multiplication of each element by 2: [2 4 6]  
Taking square of each element: [1 4 9]
```

Some other useful functions in NumPy that are heavily used for data preprocessing and feature engineering in AI are demonstrated in the following code.

```
#CODE  
  
arr = np.array([1, 2, 3, 4, 5])  
print(np.sum(arr))      # 15  
print(np.mean(arr))     # 3.0  
print(np.std(arr))      # Standard deviation  
print(np.max(arr))      # 5  
print(np.min(arr))      # 1
```

```
#For multi-dimensional arrays:
mat = np.array([[1,2,3],[4,5,6]])
print(np.sum(mat, axis=0)) # Column-wise sum    [5 7
9]
print(np.sum(mat, axis=1)) # Row-wise sum    [6 15]
```

NumPy also provides functions for different matrix operations as follows:

```
#CODE: Matrix Operations
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix multiplication
C = np.dot(A, B)
print("Matrix multiplication:", C)

# Transpose
print("Transpose of A:", A.T)

# Determinant
print("Determinant of A:", np.linalg.det(A))

# Inverse
print("Inverse of A:", np.linalg.inv(A))
```

OUTPUT:

```
Matrix multiplication:
[[19 22]
 [43 50]]
Transpose of A:
[[1 3]
 [2 4]]

Determinant of A: -2.0000000000000004

Inverse of A:
[[-2.  1.]
 [ 1.5 -0.5]]
```

AI models often require randomness for initializing weights, shuffling data, or splitting datasets. The random module of NumPy allows to generate the random data. The code for the same as follows:

```
#CODE: random module

# Random numbers between 0 and 1
print(np.random.rand(5))

# Random integers
```

```
print(np.random.randint(1, 10, size=5))

# Normal distribution (mean=0, std=1)
print(np.random.randn(5))
```

OUTPUT:

```
# Random numbers between 0 and 1
[0.89182018 0.173299    0.64496502 0.17889427
 0.94136995]
# Random integers
[6 4 8 1 3]
# Normal distribution (mean=0, std=1)
[-0.87590679 -0.24440562  0.05706411 -0.77204154 -
 0.96940411]
```

Broadcasting can be used to perform operations between arrays of different shapes. This concept can save time and memory and is commonly used in AI model computations.

```
#CODE: Broadcasting

a = np.array([[1,2,3],[4,5,6]])
b = np.array([1,2,3])

print(a + b) # array b is added to both rows of a
```

OUTPUT:

```
[[2 4 6]
 [5 7 9]]
```

AI models train better if data is normalized. Though, there is no direct function in Numpy for normalization, same can be done in one go by writing the formula.

```
#CODE: Data Normalization

data = np.array([10, 20, 30, 40, 50])
normalized = (data - np.mean(data)) / np.std(data)
print("Normalized data is ", normalized)
```

OUTPUT:

```
Normalized data is [-1.41421356 -0.70710678  0.
 0.70710678  1.41421356]
```

Designing AI applications using NumPy has a number of advantages. It provides high speed and efficiency as its array operations are significantly faster than Python lists which makes it suitable for handling large datasets. NumPy is also memory-efficient which helps in reducing the storage requirement while processing massive amounts of data. With its range of built-in mathematical and

statistical functions, it simplifies the complex computations. Moreover, NumPy is compatible with other popular AI and data science libraries such as Pandas, SciPy, TensorFlow and PyTorch.

Despite of its advantages, it does have some limitations as well. It can work only on single CPU which restricts its scalability for very large datasets. It also lacks higher-level abstractions such as models, layers and training utilities that specialized AI frameworks like TensorFlow and PyTorch provide.

11.3.2 Pandas

AI is all about making machines learn from data and make decisions. But before staring the model training, we need to organize, clean and analyse the data. This step is called data preprocessing. It is one of the important step of any machine learning model so that the model can learn the pattern from clean data. Usually, this step takes lots of time in AI projects. To help the programmers, one of the most powerful tools for preprocessing in Python is the Pandas library.

Pandas is an open-source library which provides flexible data structures and data analysis tools. It can handle large datasets, clean the missing values, merge different files and perform the statistical analysis with just a few lines of code. It acts as a bridge between the raw data and the machine learning models.

If you are using Anaconda, Pandas is already installed. Otherwise, you can install it using:

```
pip install pandas
```

To import Pandas library:

```
import pandas as pd
```

Here, pd is just an alias to make typing shorter. You can assign any name to create the alias.

The core data structures supported by Pandas are:

1. Series – A one-dimensional labeled array.
2. DataFrame – A two-dimensional labeled data structure (like a table).

A **Series** is like a column in a table. It can be defined with default index starting from 0 or with custom indices as follows:

```
#CODE: Series
import pandas as pd

data_dafult_index = pd.Series([10, 20, 30, 40])
print("Series with default indices",
data_dafult_index)
```

```
data_custom_index = pd.Series([10, 20, 30],
index=['a', 'b', 'c'])
print("Series with custom
indices",data_custom_index)
```

OUTPUT:

```
Series with default indices
0    10
1    20
2    30
3    40
dtype: int64

Series with custom indices
a    10
b    20
c    30
dtype: int64
```

Dataframe in Pandas is like a table with multiple rows and columns. It helps in handling large dataset.

```
#CODE: Dataframes

data = {
    'Name': ['P', 'Q', 'R'],
    'Age': [18, 27, 20],
    'Score': [85, 90, 88]
}

df = pd.DataFrame(data)
print(df)
```

OUTPUT:

	Name	Age	Score
0	P	18	85
1	Q	27	90
2	R	20	88

Pandas supports reading and writing data from CSV, xlsx and JSON files. The path with filename to be mentioned can be absolute path or the relative path depending upon the location of your current Python file. The code with relative path is as follows:

```
# Reading CSV file
df = pd.read_csv("data.csv")

# Writing to CSV file
df.to_csv("output.csv", index=False)
```

```
# Reading Excel file
df = pd.read_excel("data.xlsx")

# Reading JSON file
df = pd.read_json("data.json")
```

Once data is read and stored in dataframes, next step is to explore the dataset. Pandas provides a number of functions for the same. These functions help us quickly understand the dataset's structure.

```
#CODE: Data exploration

print(df.head())      # First 5 rows
print(df.tail())      # Last 5 rows
print(df.info())      # Information about dataset
print(df.shape)       # (rows, columns)
print(df.columns)     # List of column names
print(df.describe())  # Statistical summary such as
count, mean, min, max etc
```

Following code explains how to handle columns and rows in the dataframe.

```
#CODE: Data Selection

#Selecting Columns
print(df['Name'])      # Single column
print(df[['Name', 'Age']]) # Multiple columns

#Selecting Rows
print(df.iloc[0])      # First row by position
print(df.loc[1])       # Row with index label 1

#Conditional Selection
print(df[df['Age'] > 23]) # Filter rows where Age > 23
```

Real-world datasets are rarely clean. Pandas provides various functions to clean the data as follows:

```
#CODE: Data Cleaning

df.isnull().sum()      # Count missing values
df = df.dropna()       # Drop rows with missing values
df = df.fillna(0)      # Replace missing with 0
df = df.drop_duplicates() # Handling Duplicates
df['Age'] = df['Age'].astype(float) #Changing Data Types
```

AI models often require transformation of data. It can be done as follows:

```
#CODE: Data transformation
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [24, 27, 22],
    'Score': [85, 90, 88]
}
df = pd.DataFrame(data)
print("Data before transformation", df);

# Renaming Columns
df.rename(columns={'Score':'Marks'}, inplace=True)
# Sorting Data
df.sort_values('Age', ascending=False, inplace=True)
# Applying Functions
df['Marks'] = df['Marks'].apply(lambda x: x/10)

print("Data after transformation", df)
```

OUTPUT:

Data before transformation			
	Name	Age	Score
0	Alice	24	85
1	Bob	27	90
2	Charlie	22	88

Data after transformation			
	Name	Age	Marks
1	Bob	27	9.0
0	Alice	24	8.5
2	Charlie	22	8.8

Here, `inplace=True` is used in `rename` and `sort_values` function. It means that changes will be reflected in the original dataframe. As it can be seen in the output, the data gets sorted in the dataframe itself. By default, this value is false, indicating that changes are temporary. Also, `ascending=False` means that it is sorted in descending order. Further, `apply()` method is used to apply lambda function. It takes one argument, `x` and returns the value of `x` divided by 10.

Pandas provides support to concatenate, merge or group the data as follows:

```
#CODE: Data Combining

#Concatenation
df1 = pd.DataFrame({'A':[1,2], 'B':[3,4]})
df2 = pd.DataFrame({'A':[1,6], 'B':[7,8]})
result1 = pd.concat([df1, df2])
print("Concatenated Dataframe is ")
```

```
print(result1)

#Merging
result2 = pd.merge(df1, df2, on='A')
print("Merged Dataframe is ")
print(result2)
```

OUTPUT:

```
Concatenated Dataframe is
   A  B
0  1  3
1  2  4
0  1  7
1  6  8
Merged Dataframe is
   A  B_x  B_y
0  1    3    7
```

Here, `pd.concat()` function concatenates two or more dataframes based on columns. Similarly, `pd.merge` function allows us to combine 2 dataframes based on a certain column, which must be present in both dataframes. So, in the above code, the data is combined based on column “A” by fetching values corresponding to “A” from both the dataframes.

AI depends on statistics. Pandas provides built-in methods:

```
#CODE: Data Analysis

print("Average Marks:", df['Marks'].mean()) # Average
print("Median:", df['Marks'].median()) # Median
print("Standard deviation:", df['Marks'].std()) # Standard deviation
print("Correlation", df['Marks'].corr(df['Age'])) # Correlation
```

OUTPUT:

```
Average Marks: 8.766666666666667
Median: 8.8
Standard deviation: 0.2516611478423584
Correlation 0.49999999999999984
```

Pandas provides several advantages in AI by offering an easy-to-use interface with simple syntax, making the data manipulation and analysis easier. It can efficiently handle large datasets and offers flexible data structures such as Series and DataFrames for organizing and processing the information. Pandas also supports multiple data formats such as CSV, Excel and JSON which makes it useful for real-world data handling. Moreover, its strong integration with NumPy, Scikit-learn and TensorFlow ensures smooth compatibility. However, Pandas also has some limitations. It mainly works on a single machine’s

memory, which makes it challenging to handle extremely large or distributed datasets.

11.3.3 Matplotlib

AI is not just about building the models. A major part of AI projects involves understanding and visualizing the data. Visualization helps us to see the patterns, trends and relationships among data that are difficult to identify in the raw numbers. It also helps to communicate and display the results effectively to others. In Python, one of the most powerful visualization libraries is Matplotlib. It is an open-source library that allows us to create a wide range of static, animated and interactive plots. Generally, Matplotlib is used for

1. Exploratory Data Analysis (EDA) – Understanding dataset distributions, outliers, correlations and trends.
2. Model Training Insights – Plotting loss curves and accuracy during training.
3. Feature Importance – Visualizing which features contribute most to predictions.
4. Result Communication – Creating clear charts for reports and presentations.

If you are using Anaconda, Matplotlib is already installed. Otherwise, you can install it using:

```
pip install matplotlib
```

To import Matplotlib library:

```
import matplotlib.pyplot as plt
```

Here, pyplot is a submodule that provides functions to create plots and plt is the alias.

To plot a simple line

```
#CODE: Line plot

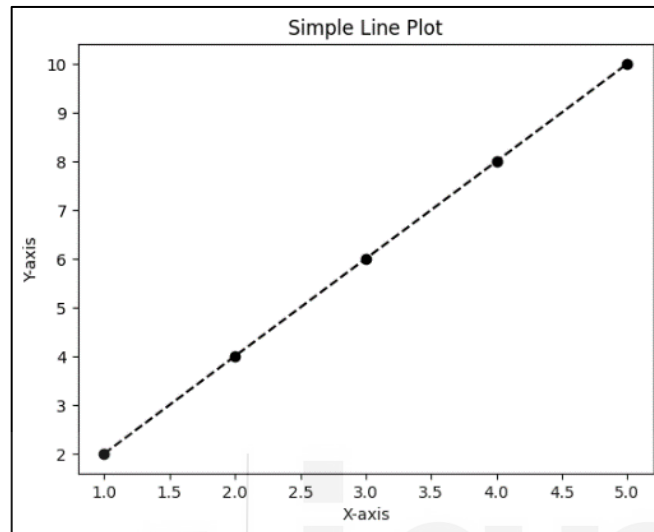
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]    #x-axis : #horizontal axis
                       (independent variable)
y = [2, 4, 6, 8, 10]  #y-axis : vertical axis
                       (dependent variable)

plt.plot(x, y, color='black', linestyle='--',
         marker='o')
#color - sets line color
#linestyle - changes line style (--, -, :)
#marker - marks data points (o, s, *)
```

```
plt.title("Simple Line Plot") #plot title
plt.xlabel("X-axis") #axis labels
plt.ylabel("Y-axis") #axis labels
plt.show()
```

OUTPUT:



Bar chart can be plotted as follows:

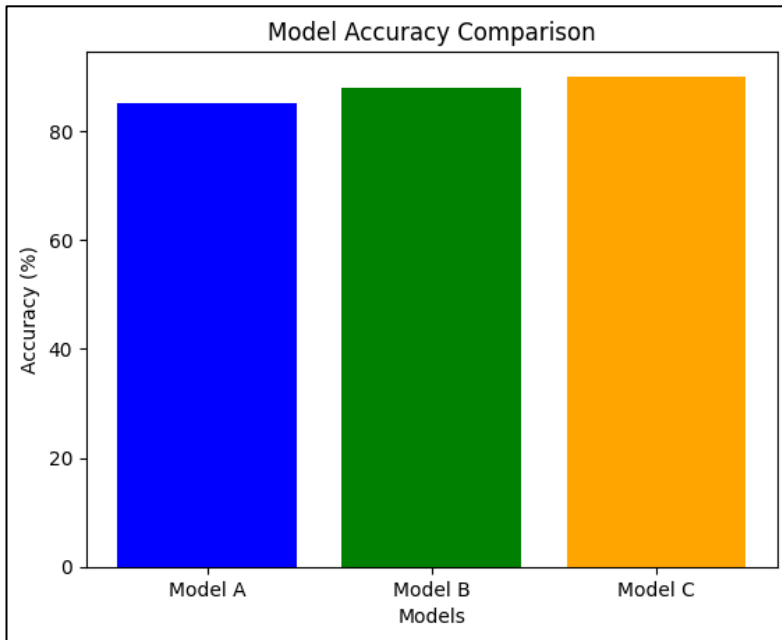
```
#CODE: Bar Chart

import matplotlib.pyplot as plt

models = ['Model A', 'Model B', 'Model C']
accuracy = [85, 88, 90]

#bar function is used for bar chart
plt.bar(models, accuracy,
color=['blue', 'green', 'orange'])
plt.title("Model Accuracy Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy (%)")
plt.show()
```

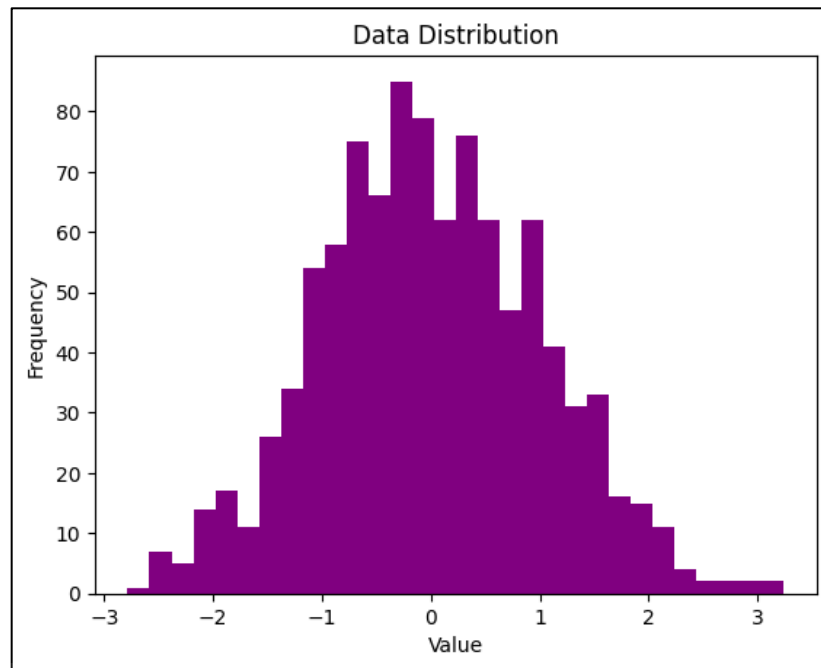
OUTPUT:



Histograms are used to show data distribution. Code is as follows:

```
#CODE: Histogram
import matplotlib.pyplot as plt
import numpy as np
data = np.random.randn(1000)

plt.hist(data, bins=30, color='purple') #hist
function is used for histogram
plt.title("Data Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```



Scatter Plot shows relationship between two variables. It is widely used in AI to study correlations.

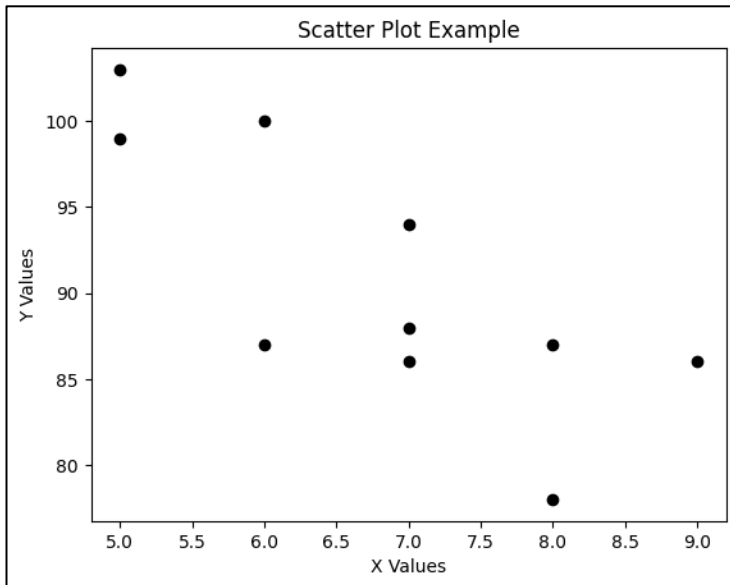
```
#CODE: Scatter Plot

import matplotlib.pyplot as plt

x = [5,7,8,7,6,9,5,6,7,8]
y = [99,86,87,88,100,86,103,87,94,78]

plt.scatter(x, y, color='black')
plt.title("Scatter Plot Example")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.show
```

OUTPUT:



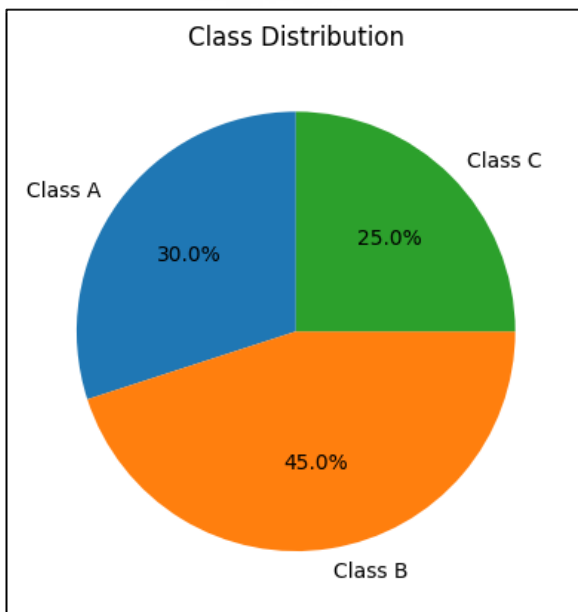
To represent proportions, Pie Charts are used as follows:

```
#CODE: Pie Chart

import matplotlib.pyplot as plt

labels = ['Class A', 'Class B', 'Class C']
sizes = [30, 45, 25]
plt.figure()
plt.pie(sizes, labels=labels, autopct='%1.1f%%',
startangle=90)
plt.title("Class Distribution")
plt.show()
```

Output:



We can display multiple plots together using subplots. They are useful in AI to visualize different aspects of the same dataset together.

```
#CODE: Subplots

import matplotlib.pyplot as plt

fig, axs = plt.subplots(2, 2) # set the frame for 4
plots arranged as 2 X 2

x = [1, 2, 3, 4, 5]      #x-axis : #horizontal axis
                          (independent variable)
y = [2, 4, 6, 8, 10]     #y-axis : vertical axis
                          (dependent variable)

axs[0, 0].plot(x, y)
axs[0, 0].set_title("Line Plot")

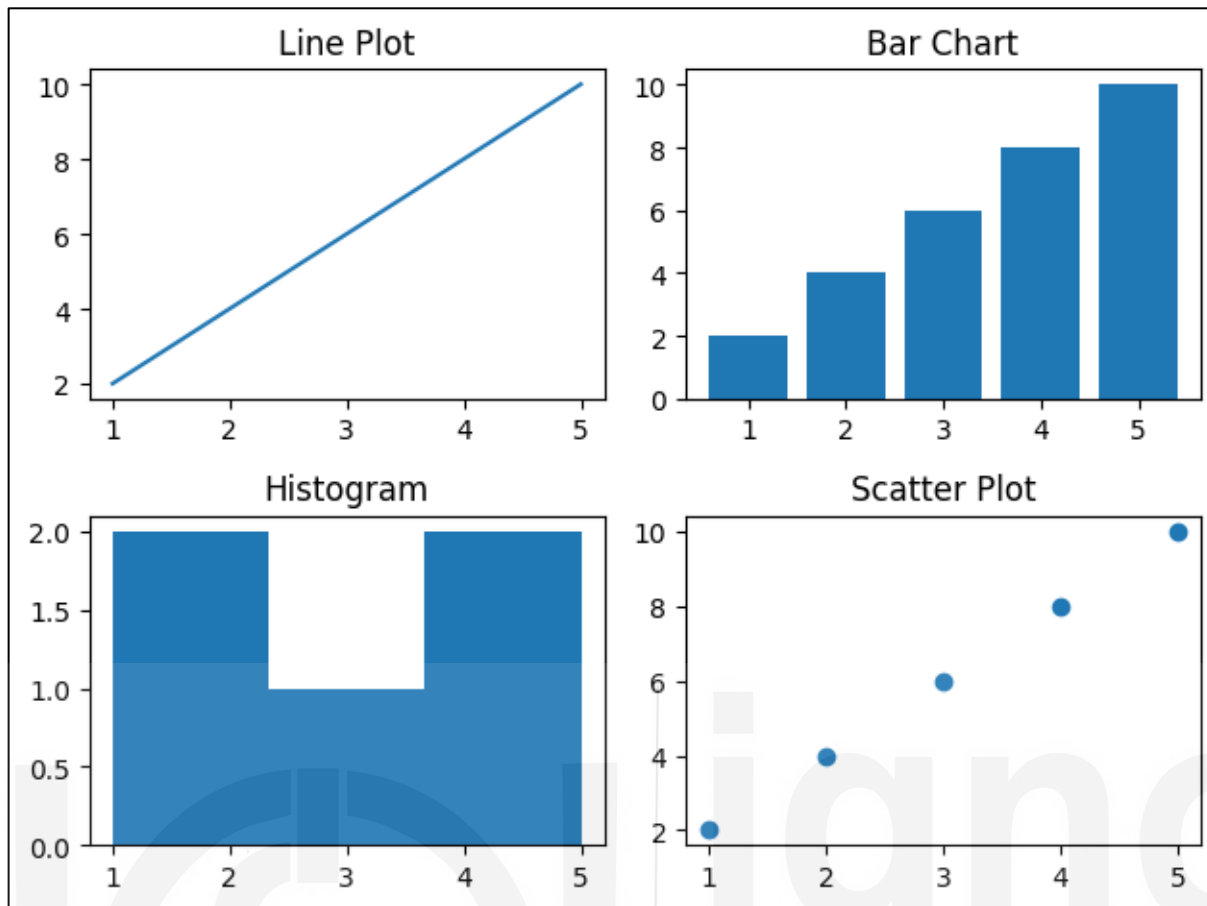
axs[0, 1].bar(x,y)
axs[0, 1].set_title("Bar Chart")

axs[1, 0].hist(x, bins=3)
axs[1, 0].set_title("Histogram")

axs[1, 1].scatter(x, y)
axs[1, 1].set_title("Scatter Plot")

plt.tight_layout()
plt.show()
```

OUTPUT;



Plots created using above codes can be saved for documentation in different formats: PNG, JPG, PDF, SVG. Code to save the plot is

```
#CODE: Save plot
plt.savefig("plot.png")
```

Matplotlib is a highly flexible visualization library that allows users to create almost any type of chart or graph, making it extremely useful for AI research and the data analysis. It can be integrated with popular Python libraries such as NumPy, Pandas, TensorFlow and PyTorch which enables the efficient handling and visualization of the complex datasets. Moreover, Matplotlib offers extensive the customization options. It allows users to modify titles, labels, styles and legends to enhance the clarity and presentation.

Despite its advantages, Matplotlib can be somewhat challenging for beginners due to its relatively complex syntax. Creating detailed or specific plots often requires writing multiple lines of code which can make the process time-consuming.

11.3.4 Scikit-learn

AI and ML are all about making computers learn from data. To do this, we need tools that make model building simple and efficient. One of the most popular and beginner-friendly libraries in Python for ML is Scikit-learn (also known as

sklearn). It is an open-source library built on top of NumPy, Pandas and Matplotlib. It provides simple and consistent functions to preprocess data, build machine learning models, evaluate performance and fine-tune results.

If you are using Anaconda, Scikit-learn is already installed. Otherwise, you can install it using:

```
pip install scikit-learn
```

To import Scikit-learn library:

```
import sklearn
```

Most of the time, we import specific modules like:

```
from sklearn.model_selection import  
train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import accuracy_score
```

Every machine learning task in Scikit-learn usually follows these steps:

1. Load Data
2. Split Data into training and testing sets
3. Preprocess Data (cleaning, scaling, encoding)
4. Choose Model (e.g., Linear Regression, Decision Tree)
5. Train Model using training data
6. Evaluate Model using testing data
7. Fine-tune if necessary

Code for Linear Regression (Supervised Learning) to predicts continuous values is as follows:

```
#CODE: Linear Regression  
  
import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
  
# Sample data  
data = {  
    'Hours_Studied': [1,2,3,4,5,6,7,8],  
    'Marks': [30,35,50,55,65,70,75,85]  
}  
df = pd.DataFrame(data)  
  
# Features and target  
X = df[['Hours_Studied']] # Independent variable  
y = df['Marks']           # Target variable
```

```

# Train-test split
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

print("Predicted Marks:", y_pred)

```

This Python code demonstrates a simple example of Linear Regression using the scikit-learn library. It first creates a sample dataset containing two columns: Hours_Studied (independent variable) and Marks (dependent variable). The dataset is then converted into a pandas DataFrame. The feature (X) is defined as the number of study hours and the target (y) is the corresponding marks. The data is split into training and testing sets using train_test_split(), where 80% is used for training and 20% for testing. A LinearRegression model is then created and trained on the training data using the fit() method. After training, the model predicts marks for the test data (X_test) and the predicted values (y_pred) are printed, showing how well the model estimates marks based on hours studied.

OUTPUT:

```
Predicted Marks: [54.16666667 85.5952381 ]
```

Code for classification with Decision Tree to predicts categories as follows:

```

#CODE: Decison Tree

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Sample data
data = {
    'Age': [18, 19, 20, 21, 22, 23],
    'Score': [55, 60, 65, 70, 75, 80],
    'Passed': [0, 0, 0, 1, 1, 1]
}
df = pd.DataFrame(data)

X = df[['Age', 'Score']]
y = df['Passed']

# Train-test split

```

```

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.3)

# Train model
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)

# Predict
y_pred = clf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

```

This Python code demonstrates the use of a Decision Tree Classifier for predicting whether a student has passed or not based on their Age and Score. It begins by creating a small dataset with three columns: Age, Score and Passed (where 1 indicates pass and 0 indicates fail) and stores it in a pandas DataFrame. The features (X) are taken as Age and Score, while the target variable (y) is Passed. The dataset is split into training and testing sets using `train_test_split()`, with 70% used for training and 30% for testing. A `DecisionTreeClassifier` model is then created and trained on the training data using the `fit()` method. After training, the model predicts outcomes for the test data (X_test) using `predict()`. Finally, the accuracy of the model is calculated using `accuracy_score()` and printed, showing how well the decision tree predicts student pass/fail outcomes.

Code for clustering with K-Means to group similar data as follows:

```

#CODE: Clustering : K-Means

import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Random dataset
X =
np.array([[1,2],[1,4],[1,0],[10,2],[10,4],[10,0]])

# Train K-Means model
kmeans = KMeans(n_clusters=2, random_state=0).fit(X)

print("Cluster Centers:", kmeans.cluster_centers_)

# Plot
plt.scatter(X[:,0], X[:,1], c=kmeans.labels_)
plt.scatter(kmeans.cluster_centers_[0,0],
kmeans.cluster_centers_[0,1], marker='x',
color='red')
plt.show()

```

This Python code illustrates K-Means clustering, an unsupervised learning technique used to group similar data points. It begins by creating a small random dataset X containing six two-dimensional points. The KMeans algorithm from scikit-learn is then used to divide the data into two clusters (n_clusters=2). The model is trained using the fit() method, which calculates the optimal positions of the cluster centers. The coordinates of these centers are printed using kmeans.cluster_centers_. Finally, the data points are visualized with matplotlib, where each point is colored according to its assigned cluster (kmeans.labels_) and the cluster centers are marked with red “X” symbols, clearly showing the two formed groups.

Choosing the right model settings (hyperparameters) in ML model improves the accuracy. Sklearn provides a function to search for best parameters.

```
#CODE: Best parameter search

from sklearn.model_selection import GridSearchCV

params = {'max_depth':[2,3,4]}
clf = DecisionTreeClassifier()
grid = GridSearchCV(clf, params, cv=3)
grid.fit(X, y)

print("Best Parameters:", grid.best_params_)
```

This Python code shows how to perform hyperparameter tuning using GridSearchCV to find the optimal configuration for a machine learning model. Here, a DecisionTreeClassifier is created and a parameter grid params is defined with different values of max_depth (2, 3 and 4), which controls how deep the decision tree can grow. The GridSearchCV function from scikit-learn runs the model multiple times—once for each combination of parameters—using 3-fold cross-validation (cv=3) to evaluate model performance. It then fits the model on the dataset (X, y) and automatically selects the parameter value that produces the best results. Finally, grid.best_params_ prints the best parameter setting (in this case, the most suitable max_depth) for achieving the highest model accuracy.

Scikit-learn is a beginner-friendly library with simple and easy-to-understand syntax. It is suitable for newcomers to machine learning. It is a comprehensive library that offers almost all classical machine learning algorithms such as regression, classification, clustering and model selection. The library is reliable and widely used for both academia and industry for research and application development. It can be integrated with other popular Python libraries like NumPy, Pandas and various visualization tools.

Despite its strengths, Scikit-learn is not designed for deep learning applications, where frameworks like TensorFlow or PyTorch are preferred. It works best for small to medium-sized datasets while handling very large datasets which may require distributed or GPU-based systems like Spark ML. Additionally, Scikit-learn has limited support for online or real-time learning which makes it less suitable for scenarios requiring continuous model updates.

11.3.5. TensorFlow

TensorFlow is an open-source machine learning and deep learning framework developed by Google Brain in 2015. It is one of the most widely used libraries for building and training AI models. The name “TensorFlow” comes from the concept of tensors (multidimensional data arrays) that flow through a system of mathematical operations.

In TensorFlow, data is represented as tensors. A tensor is a generalization of vectors and matrices to higher dimensions.

- A scalar (a single number) is a 0-D tensor.
- A vector (a list of numbers) is a 1-D tensor.
- A matrix (rows and columns) is a 2-D tensor.
- Higher dimensions are possible for more complex data.

Example to creates a 2x2 matrix (a rank-2 tensor)

```
#CODE: Tensor Flow Example

import tensorflow as tf
x = tf.constant([[1, 2], [3, 4]])
print(x)
```

OUTPUT:

```
tf.Tensor(
[[1 2]
 [3 4]], shape=(2, 2), dtype=int32)
```

TensorFlow uses the idea of a computation graph to represent mathematical operations. In a computation graph, nodes represent operations (like addition, multiplication) and edges represent the tensors (data) that flow between operations.

In early versions of TensorFlow (before version 2.0), users had to first define the graph and then run it in a session. This was called a static graph, which was powerful but difficult to debug. From TensorFlow 2.0 onward, TensorFlow introduced eager execution, which means operations run immediately as they are called — similar to how Python works. This made TensorFlow much easier and more intuitive, like PyTorch.

TensorFlow provides a high-level API called Keras to make building neural networks simple and user-friendly. A neural network is made up of layers — such as input, hidden and output layers — that process data step by step to make predictions.

Example:

```
#CODE: Neural Network Example
```

```

from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Dense(16, activation='relu',
input_shape=(10,)),
    layers.Dense(1, activation='sigmoid')
])

model.summary()

```

OUTPUT:

Model: "sequential_3"

Layer (type) Param #	Output Shape
dense_6 (Dense) 176	(None, 16)
dense_7 (Dense) 17	(None, 1)
Total params: 193 (772.00 B)	
Trainable params: 193 (772.00 B)	
Non-trainable params: 0 (0.00 B)	

Here, Sequential creates a linear stack of layers. The Dense layer is a fully connected layer used in many neural networks. TensorFlow automatically handles complex computations like backpropagation and gradient descent during training.

To compile and train the model:

```

model.compile(optimizer='adam',
loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10)

```

11.3.6 PyTorch

After TensorFlow, another powerful and easy-to-use deep learning framework was developed: PyTorch, created by Facebook's AI Research (FAIR) team in 2016. Like TensorFlow, PyTorch is used for building, training and deploying deep learning models. However, PyTorch focuses more on simplicity, flexibility

and dynamic computation, which makes it very popular among researchers, students and developers.

TensorFlow (especially in its early versions) used static computation graphs, meaning you had to define the entire model before running it. PyTorch, on the other hand, uses a dynamic computation graph that means it builds the graph as you go. This makes PyTorch feel more like regular Python code, which is easier to write, understand and debug.

The basic building block in PyTorch is the tensor, similar to NumPy arrays but faster and compatible with GPUs. Tensors are used to store and manipulate data.

```
#CODE: Pytorch Example

import torch
x = torch.tensor([[1, 2], [3, 4]])
print(x)
print(x * 2)    # Tensor operations
```

OUTPUT:

```
tensor([[1, 2],
        [3, 4]])
tensor([[2, 4],
        [6, 8]])
```

PyTorch provides autograd, a feature that automatically calculates gradients (derivatives) for optimization. It is mainly used in training neural networks through backpropagation.

```
#CODE: Pytorch Example 2

x = torch.tensor(2.0, requires_grad=True)
y = x ** 3 + 4 * x
y.backward()    # Compute gradient
print(x.grad)   # dy/dx = 3x2 + 4
```

Here, PyTorch automatically computes the gradient of y with respect to x , which is very useful for updating model parameters during training.

PyTorch provides a powerful module called `torch.nn` that helps define and train neural networks easily. It includes layers, activation functions and loss functions.

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(2, 3),    # Input layer: 2 neurons →
    nn.ReLU(),          # Activation function
    Hidden layer: 3 neurons
```

```
nn.Linear(3, 1)    # Output layer
)
print(model)
```

Check your Progress 1

1. Write Python program using matplotlib to draw a pie chart for your marks in 5 subjects.

.....

.....

.....

2. Write a python code to do statistical analysis (average, min, max, standard deviation) of your marks as mentioned in previous question.

.....

.....

.....

3. What are tensors and how they help us to write program for deep leaning?

.....

.....

.....

11.4 HANDS ON IMPLEMENTATION

Following section discusses the Python implementation of various well-known problems in AI-domain such as: N-Queen Problem, Min-Max Algorithm and Alpha-Beta Pruning, Breadth First Search, Depth First Search, Best First Search, A* Algorithm, AO* Algorithm and Iterative Deepening A*.

11.4.1 N-Queen Problem

The N-Queen problem is a classic backtracking problem where the goal is to place N queens on an N×N chessboard such that no two queens attack each other. Since queens in chess can attack along rows, columns and diagonals, the challenge is to arrange them in a safe way. The Python program solves this by using a method called backtracking. This means it tries to place a queen in the first row, checks if it is safe and then moves to the next row. If it finds a row where no safe place is available, it goes back (backtracks) and changes the position of the previous queen. This process continues until all queens are placed correctly. For more details about N-queen problem, refer to section 2.4.

Following is the code to solve N-queen problem:

```
#CODE; N-Queen Problem
```

```

def print_solution(board): #function to print the
solution
    n = len(board)
    for i in range(n):
        row = ""
        for j in range(n):
            if board[i][j] == 1:
                row += "Q "
            else:
                row += ". "
        print(row)
    print("\n")

def is_safe(board, row, col):
#Function to check if a particular position is safe
to place the next queen
    n = len(board)

    # Check column
    for i in range(row):
        if board[i][col] == 1:
            return False

    # Check upper-left diagonal
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check upper-right diagonal
    i, j = row, col
    while i >= 0 and j < n:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1

    return True

def solve_nqueens(board, row): #Function to solve N-
queen problem
    n = len(board)
    if row == n: # All queens are placed
        print_solution(board)
        return True

    res = False
    for col in range(n):

```

```

        if is_safe(board, row, col):
            board[row][col] = 1 # Place queen
            res = solve_nqueens(board, row + 1) or
res #recursive function call
            board[row][col] = 0 # Backtrack

    return res

def n_queens(n):
    board = [[0] * n for _ in range(n)]
    if not solve_nqueens(board, 0):
        print("No solution exists")

# Example: Solve for N=4
n_queens(4)

```

OUTPUT:

```

. Q . .
. . . Q
Q . . .
. . Q .

. . Q .
Q . . .
. . . Q
. Q . .

```

Here, Q represents a queen and . represents an empty cell. It prints **all possible solutions** for the N-Queen problem.

In the Python code, N-Queen problem is solved using recursion. For each row, the program tries placing a queen in every column and before placing, it checks whether the position is safe. The `is_safe` function ensures that there is no queen in the same column, or along the left and right diagonals above the current row. If the position is safe, the queen is placed and the algorithm recursively moves to the next row. If at any point no safe position is found, it backtracks by removing the previously placed queen and trying another column. Once all queens are successfully placed, the configuration is stored as a valid solution. After finding solutions, the code prints the total number of them and visualizes each one on a chessboard. For example, when $n=4$, the program finds 2 valid solutions and displays both boards with queens arranged so that none can attack each other.

11.4.2 Min-Max Algorithm and Alpha-Beta Pruning

The Min-Max algorithm is a decision-making algorithm used in two-player games where one player tries to maximize the score (Maximizer) while the other tries to minimize it (Minimizer). The idea is that both players play optimally. The algorithm builds a game tree of all possible moves. At each step, if it is the

Maximizer's turn, the algorithm chooses the move with the highest possible value, assuming the opponent will respond in the best way to minimize the score. On the other hand, if it is the Minimizer's turn, it chooses the move with the lowest possible value, assuming the opponent will try to maximize their advantage later. This process continues until a terminal state is reached (win, lose, or draw) and values are assigned to the outcomes. The values then backtrack up the tree and the root player picks the move that guarantees the best result against an optimal opponent. For example, in tic-tac-toe, the Min-Max algorithm can simulate all possible moves and ensure that the AI either wins or forces a draw, but never loses. Refer to Section 2.6 for more details about Min-Max algorithm. Here is the code to play tic-tac-toe using Min-Max approach.

```
#CODE: Min-Max Algorithm for Tic-Tac-Toe Game

import math

# Check if there are moves left on the board
def is_moves_left(board):
    for row in board:
        if "_" in row:
            return True
    return False

# Evaluate the board
def evaluate(board):
    # Check rows
    for row in board:
        if row.count("X") == 3:
            return 10
        elif row.count("O") == 3:
            return -10

    # Check columns
    for col in range(3):
        if board[0][col] == board[1][col] == board[2][col]:
            if board[0][col] == "X":
                return 10
            elif board[0][col] == "O":
                return -10

    # Check diagonals
    if board[0][0] == board[1][1] == board[2][2]:
        if board[0][0] == "X":
            return 10
        elif board[0][0] == "O":
            return -10
    if board[0][2] == board[1][1] == board[2][0]:
        if board[0][2] == "X":
            return 10
        elif board[0][2] == "O":
            return -10
```

```

        return -10

    return 0

# Minimax function
def minimax(board, depth, is_max):
    score = evaluate(board)

    # If Maximizer has won
    if score == 10:
        return score - depth # prefer faster wins

    # If Minimizer has won
    if score == -10:
        return score + depth # prefer slower losses

    # If no moves left => draw
    if not is_moves_left(board):
        return 0

    if is_max:
        best = -math.inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == "_":
                    board[i][j] = "X"
                    best = max(best, minimax(board,
depth + 1, False))
                    board[i][j] = "_"
            return best
        else:
            best = math.inf
            for i in range(3):
                for j in range(3):
                    if board[i][j] == "_":
                        board[i][j] = "O"
                        best = min(best, minimax(board,
depth + 1, True))
                        board[i][j] = "_"
            return best

# Find the best move
def find_best_move(board):
    best_val = -math.inf
    best_move = (-1, -1)

    for i in range(3):
        for j in range(3):
            if board[i][j] == "_":
                board[i][j] = "X"
                move_val = minimax(board, 0, False)

```

```

        board[i][j] = "_"
        if move_val > best_val:
            best_move = (i, j)
            best_val = move_val

    return best_move

# Example Board
board = [
    ["X", "O", "X"],
    ["O", "O", "_"],
    ["_", "_", "_"]
]

best_move = find_best_move(board)
print("The AI should place at:", best_move)

```

This Python program uses the Min-Max algorithm to decide the best move for the AI in Tic-Tac-Toe. The board is represented as a 3×3 list, where "X" is the AI, "O" is the opponent and "_" represents an empty cell. The evaluate function assigns a score to the board: +10 if the AI wins, -10 if the opponent wins and 0 for a draw. The minimax function then simulates all possible future moves. If it is the AI's turn (Maximizer), it tries to maximize the score by choosing the move that leads to the best outcome. If it is the opponent's turn (Minimizer), it chooses moves that minimize the score, making the game harder for the AI. To make decisions smarter, the depth of the recursion is included in the score, so the AI prefers to win quickly and delay losses as much as possible. Finally, the find_best_move function loops through all available empty spaces, uses minimax to evaluate them and picks the move with the highest score. When we run the program on a given board, it prints the best position for the AI to play next. This ensures the AI always plays optimally: it will win when possible, or at least force a draw.

While the algorithm guarantees the best result, it can be slow for large games because it explores all possible moves. To make it efficient, a technique called Alpha-Beta Pruning is often added to cut off unnecessary branches in the game tree. It is explained in detailed in Section 2.7.

```

#CODE: Alpha-beta Pruning for Tic-Tac-Toe Game

import math

# Check if moves left
def is_moves_left(board):
    for row in board:
        if "_" in row:
            return True
    return False

# Evaluate the board
def evaluate(board):

```

```

# Check rows
for row in board:
    if row.count("X") == 3:
        return 10
    elif row.count("O") == 3:
        return -10

# Check columns
for col in range(3):
    if board[0][col] == board[1][col] ==
board[2][col]:
        if board[0][col] == "X":
            return 10
        elif board[0][col] == "O":
            return -10

# Check diagonals
if board[0][0] == board[1][1] == board[2][2]:
    if board[0][0] == "X":
        return 10
    elif board[0][0] == "O":
        return -10
if board[0][2] == board[1][1] == board[2][0]:
    if board[0][2] == "X":
        return 10
    elif board[0][2] == "O":
        return -10

return 0

# Minimax with Alpha-Beta Pruning
def minimax(board, depth, alpha, beta, is_max):
    score = evaluate(board)

    # If terminal state
    if score == 10:
        return score - depth
    if score == -10:
        return score + depth
    if not is_moves_left(board):
        return 0

    if is_max: # Maximizer (AI "X")
        best = -math.inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == "_":
                    board[i][j] = "X"
                    val = minimax(board, depth + 1,
alpha, beta, False)
                    board[i][j] = "_"

```

```

        best = max(best, val)
        alpha = max(alpha, best) #
Setting the value for alpha
        if beta <= alpha: # Prune
branch
            break

        return best
    else: # Minimizer (Opponent "O")
        best = math.inf
        for i in range(3):
            for j in range(3):
                if board[i][j] == "_":
                    board[i][j] = "O"
                    val = minimax(board, depth + 1,
alpha, beta, True)
                    board[i][j] = "_"
                    best = min(best, val)
                    beta = min(beta, best) # Setting
the value for beta
                if beta <= alpha: # Prune
branch
                    break

            return best

# Find the best move
def find_best_move(board):
    best_val = -math.inf
    best_move = (-1, -1)

    for i in range(3):
        for j in range(3):
            if board[i][j] == "_":
                board[i][j] = "X"
                move_val = minimax(board, 0, -
math.inf, math.inf, False)
                board[i][j] = "_"
                if move_val > best_val:
                    best_move = (i, j)
                    best_val = move_val

    return best_move

# Example Board
board = [
    ["X", "O", "X"],
    ["O", "O", "_"],
    ["_", "_", "_"]
]

best_move = find_best_move(board)
print("The AI should place at:", best_move)

```

This program uses the Min-Max algorithm with Alpha-Beta Pruning to play Tic-Tac-Toe optimally. The AI is "X" (Maximizer) and the opponent is "O" (Minimizer). The evaluate function scores a board: +10 if AI wins, -10 if opponent wins and 0 if it's a draw. The minimax function explores possible moves recursively: the AI tries to maximize the score, while the opponent tries to minimize it. Alpha (best already guaranteed for Maximizer) and Beta (best already guaranteed for Minimizer) are used to prune (cut off) unnecessary branches of the game tree. If a branch cannot possibly improve the final decision, the algorithm skips it, saving time. The find_best_move function checks all empty spots, runs minimax with pruning and picks the move with the highest value. The result is that the AI always makes the optimal move, but faster than regular Min-Max, since many losing branches are ignored.

Check your Progress 2

1. What is the purpose of the is_safe() function in the N-Queen implementation?

.....

.....

.....

2. In the backtracking solution of the N-Queen problem, what happens when a position is not safe?

.....

.....

.....

3. In Minimax implementation, what is the base case for recursion?

.....

.....

.....

11.4.3 Breadth First Search

Breadth-First Search (BFS) (explained in Section 3.3.1) is a graph traversal algorithm that explores nodes level by level. Refer to the graph given in figure 1, BFS starts from a given node (here "A") and visits all its direct neighbors before moving on to their neighbors. In the program, a queue is used to ensure nodes are processed in the correct order. First, "A" is added to the queue. Then, its neighbors "B" and "C" are visited and enqueued. Next, BFS explores "B", which leads to "D" and "E". Then it processes "C", leading to "F". Finally, "D", "E" and "F" are visited, but since "F" was already added earlier, it is not revisited. The traversal order becomes ['A', 'B', 'C', 'D', 'E', 'F']. BFS is very useful for finding the shortest path in an unweighted graph, or for exploring networks like social connections, web crawlers and AI game moves.

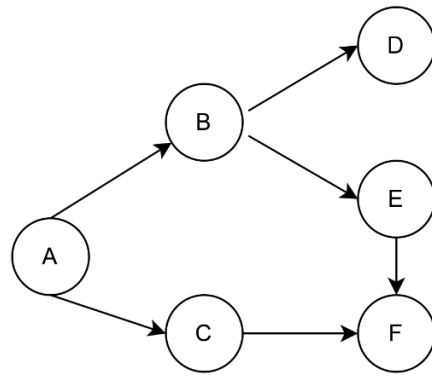


Figure 1: Sample graph

```

#CODE: BFS

from collections import deque

# BFS function
def bfs(graph, start):
    visited = set()          # To keep track of
    visited nodes           # visited nodes
    queue = deque([start])   # Queue for BFS
    order = []               # To store traversal
    order

    while queue:
        node = queue.popleft()
        if node not in visited:
            visited.add(node)
            order.append(node)

            # Add neighbors to queue
            for neighbor in graph[node]:
                if neighbor not in visited:
                    queue.append(neighbor)

    return order

# Example Graph (Adjacency List)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Run BFS
result = bfs(graph, 'A')
print("BFS Traversal:", result)

```

OUTPUT:

```
BFS Traversal: ['A', 'B', 'C', 'D', 'E', 'F']0
```

11.4.4 Depth First Search

Depth-First Search (DFS) (explained in Section 3.3.4) is a graph traversal algorithm that explores as far as possible along one path before backtracking. In this program, DFS (refer figure 1) starts from node "A". First, it visits "A", then moves to its first neighbor "B". From "B", it goes to "D", which has no further neighbors, so it backtracks. Next, it visits "E" (another neighbor of "B") and from "E", it proceeds to "F". Once "F" is done, DFS backtracks again to explore any remaining nodes. Finally, it visits "C" (the other neighbor of "A"), but since "F" was already visited, it does not repeat it. The traversal order is ['A', 'B', 'D', 'E', 'F', 'C']. Unlike BFS, which explores level by level, DFS goes deep into the graph before backtracking, making it useful for tasks like maze solving, pathfinding and topological sorting.

```
#CODE: DFS

# DFS function using recursion
def dfs(graph, node, visited=None, order=None):
    if visited is None:
        visited = set()
    if order is None:
        order = []

    if node not in visited:
        visited.add(node)
        order.append(node)

        # Visit all neighbors
        for neighbor in graph[node]:
            dfs(graph, neighbor, visited, order)

    return order

# Example Graph (Adjacency List)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Run DFS
result = dfs(graph, 'A')
print("DFS Traversal:", result)
```

OUTPUT:

```
DFS Traversal: ['A', 'B', 'D', 'E', 'F', 'C']
```

11.4.5 Best First Search

Best First Search (explained in Section 3.7.3) is a graph search algorithm that uses a heuristic value (represented by h) to decide which node to explore first. Heuristic value is an estimate of closeness of the current node to the goal. Unlike normal BFS or DFS, it doesn't explore blindly; instead, it chooses the path that looks most promising based on the heuristic value. In this program, we use a priority queue to always expand the node with the smallest heuristic value. Refer to figure 2, consider start node as A and goal node as F. Starting from "A", the algorithm looks at neighbors "B" ($h=4$) and "C" ($h=3$). Since "C" has a lower heuristic, it is explored first. From "C", we reach "F" directly. Since "F" is the goal, the search stops. The final path is ['A', 'C', 'F']. This makes Best First Search faster than normal BFS or DFS when a good heuristic is available, though it does not always guarantee the shortest path (unlike A*). It is often used in AI pathfinding and puzzle-solving where an informed guess can save a lot of time.

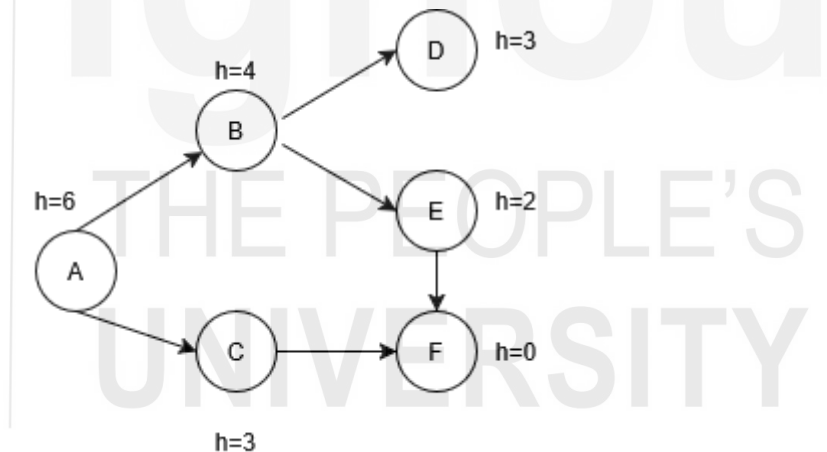


Figure 2: Sample graph for Best First Search

```
#CODE: Best First Search

import heapq

# Best First Search Algorithm
def best_first_search(graph, heuristic, start, goal):
    visited = set()
    pq = [] # priority queue (min-heap)
    heapq.heappush(pq, (heuristic[start], start)) # (heuristic, node)
    path = []
```

```

while pq:
    heuristic, node = heapq.heappop(pq)

    if node not in visited:
        visited.add(node)
        path.append(node)

        if node == goal:
            return path

        for neighbor in graph[node]:
            if neighbor not in visited:
                heapq.heappush(pq,
(heuristic[neighbor], neighbor))

    return path

# Example Graph with heuristic values
# Example Graph (Adjacency List)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

heuristic = {
    'A': 6,
    'B': 4,
    'C': 3,
    'D': 5,
    'E': 2,
    'F': 0    # Goal node
}

# Run Best First Search
start, goal = 'A', 'F'
result = best_first_search(graph, heuristic, start,
goal)
print(f"Best First Search Path from {start} to
{goal}:", result)

```

OUTPUT:

```
Best First Search Path from A to F: ['A', 'C', 'F']
```

11.4.6 A* Algorithm

A* Search (explained in section 3.8) is one of the most powerful and widely used search algorithms in AI because it always finds the shortest path if the heuristic is correct. It combines the strengths of Dijkstra's Algorithm and Best First Search. In this program, each node keeps two values:

- $g(n)$ = actual cost from the start node to the current node.
- $h(n)$ = heuristic estimate from the current node to the goal.

Finally, total estimated cost is calculated as $f(n) = g(n) + h(n)$

The algorithm uses a priority queue to always expand the node with the lowest $f(n)$. Starting at "A", it explores "B" ($f(n) = 4+2= 6$) and "C" ($f(n) = 3+1= 4$). It selects C as it has a lower overall $f(n)$ because of its heuristic. From "C", the goal "F" is quickly reached, giving the optimal path ['A', 'C', 'F']. This shows how A* balances between exploring the cheapest path so far ($g(n)$) and the most promising direction ($h(n)$), making it efficient and optimal. A* is widely used in maps (Google Maps, GPS routing), robotics and puzzle-solving because it guarantees the shortest path when the heuristic is admissible.

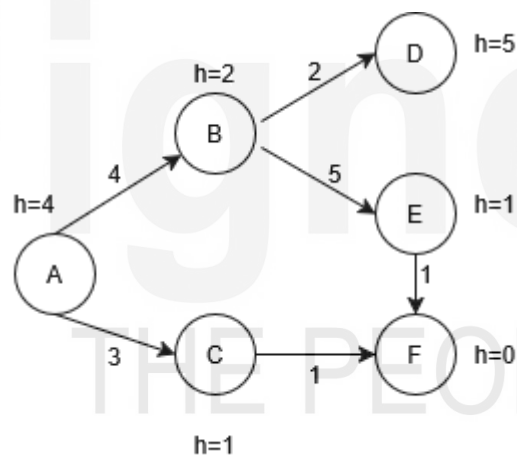


Figure 3: Sample graph for A* algorithm

```

#CODE: A* Search

import heapq

# A* Search Algorithm
def a_star_search(graph, heuristics, start, goal):
    pq = [] # priority queue (min-heap)
    heapq.heappush(pq, (0, start)) # (f = g+h,
    node)

    parent = {start: None} # to reconstruct path
    g_cost = {start: 0} # cost from start to
    current node

    while pq:
        f, node = heapq.heappop(pq)
  
```

```

        if node == goal:
            # reconstruct path
            path = []
            while node:
                path.append(node)
                node = parent[node]
            return path[::-1]

    for neighbor, cost in graph[node]:
        tentative_g = g_cost[node] + cost

        if neighbor not in g_cost or tentative_g
< g_cost[neighbor]:
            g_cost[neighbor] = tentative_g
            f_cost = tentative_g +
heuristics[neighbor]
            heapq.heappush(pq, (f_cost,
neighbor))
            parent[neighbor] = node

    return None

# Example Graph (with edge costs)
graph = {
    'A': [('B', 4), ('C', 3)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 1)],
    'D': [],
    'E': [('F', 1)],
    'F': []
}

# Heuristic values (straight-line estimate to goal
'F')
heuristics = {
    'A': 4,
    'B': 2,
    'C': 1,
    'D': 5,
    'E': 1,
    'F': 0
}

# Run A* Search
start, goal = 'A', 'F'
result = a_star_search(graph, heuristics, start,
goal)
print(f"A* Path from {start} to {goal}:", result)

```

OUTPUT:

11.4.7 AO* Algorithm

The AO* (And-Or Star) algorithm (explained in Section 3.9) is used in AND-OR graphs (as shown in Figure 4) for problem solving. Unlike A* or BFS/DFS, which work on simple paths, AO* works where solutions require satisfying multiple subgoals together (AND nodes) or choosing between alternatives (OR nodes).

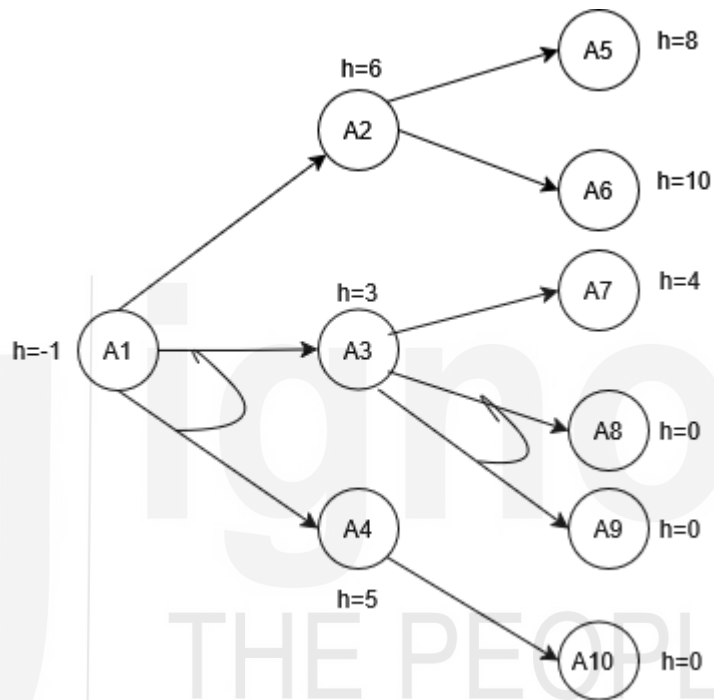


Figure 4: Sample graph for AO* algorithm

```
#CODE: AO* Search
```

```
#function to calculate the cost of each node
def cal_cost(H, cd, wt = 1):
    cost = {}
    if 'AND' in cd:
        nodes1 = cd['AND']
        andpath = ' AND '.join(nodes1)
        finalandpath = sum(H[node]+wt for node in
nodes1)
        cost[andpath] = finalandpath

    if 'OR' in cd:
        nodes2 = cd['OR']
        orpath = ' OR '.join(nodes2)
        finalorpath = min(H[node]+wt for node in
nodes2)
        cost[orpath] = finalorpath
    return cost
```

```

# function for cost updation
def update_cost(H, graph, wt=1):
    nodes = list(graph.keys())
    nodes.reverse()
    mincost = {}
    for key in nodes:
        cd = graph[key]
        print(key, ':', graph[key], '->',
cal_cost(H, cd, wt))
        c = cal_cost(H, cd, wt)
        H[key] = min(c.values())
        mincost[key] = cal_cost(H, cd, wt)
    return mincost

# function to print the shortest path
def shortest_path(start, Updated_cost, H):
    path = start
    if start in Updated_cost.keys():
        Min_cost = min(Updated_cost[start].values())
        key = list(Updated_cost[start].keys())
        values = list(Updated_cost[start].values())
        Index = values.index(Min_cost)

        nn = key[Index].split()

        if len(nn) == 1:
            start = nn[0]
            path += '<<-' + shortest_path(start,
Updated_cost, H)
        else:
            path += '<<-( '+key[Index]+' ) '
            start = nn[0]
            path += '[' + shortest_path(start,
Updated_cost, H) + ' + '
            start = nn[-1]
            path += shortest_path(start,
Updated_cost, H) + ']'

    return path

# Sample
H = {'A1': -1, 'A2': 6, 'A3': 3, 'A4': 5, 'A5': 8,
'A6': 10, 'A7': 4, 'A8': 0, 'A9': 0, 'A10': 0}

graph = {
    'A1': {'OR': ['A2'], 'AND': ['A3', 'A4']},
    'A2': {'OR': ['A5', 'A6']},
    'A3': {'OR': ['A7'], 'AND': ['A8', 'A9']},
    'A4': {'OR': ['A10']}

```

```

}

# Weight of each edge is considered as 1
wt = 1

Updated_cost = update_cost(H, graph, wt)
print('Shortest Path:\n', shortest_path('A1',
Updated_cost, H))

```

OUTPUT:

```

A4 : {'OR': ['A10']} -> {'A10': 1}
A3 : {'OR': ['A7'], 'AND': ['A8', 'A9']} -> {'A8 AND
A9': 2, 'A7': 5}
A2 : {'OR': ['A5', 'A6']} -> {'A5 OR A6': 9}
A1 : {'OR': ['A2'], 'AND': ['A3', 'A4']} -> {'A3 AND
A4': 5, 'A2': 10}
Shortest Path:
A1<<-(A3 AND A4) [A3<<-(A8 AND A9) [A8 + A9] + A4<<-
A10]

```

Here, weight of all edges is considered to be 1. For the given graph, first path from A1 -> A2 and A1->(A3+A4) is explored which is 7 and 10 respectively. So, A1 -> A2 is followed. Then, A2-> A5 and A2-> A6 is explored and weight of A2 is updated be min (9, 10). Then possible paths from a A3 are explored and its weight is updated as 2. Similarly, weight of A4 is updated to 1 after path exploration. Finally weight of A1 is updated considering updated weights of A2, A3 and A4.

11.4.8 Iterative Deepening A*

The IDA* algorithm (explained in 3.10.1) combines the optimality of A* with the space efficiency of DFS. It begins with a cost limit equal to the heuristic of the start node. Then it performs a depth-first search but only expands nodes where the total cost $f(n) = g(n) + h(n)$ is less than or equal to the current limit. If the goal is not reached, the limit is increased to the smallest $f(n)$ that exceeded the previous bound and the search is restarted. In the graph given in figure 3, the algorithm starts from "A" with a bound of 4. It first tries exploring "B" and "C", but the path "A → C → F" is found within the bound and is chosen as the optimal path. This method avoids storing all frontier nodes like A*, thus saving memory while still guaranteeing an optimal solution. IDA* is widely used in puzzle-solving (like 8-puzzle, 15-puzzle) and in AI search problems where memory is critical.

```

#CODE: Iterative Deepening A*

graph = {
    'A': [('B', 4), ('C', 3)],
    'B': [('D', 2), ('E', 5)],

```

```

    'C': [('F', 1)],
    'D': [],
    'E': [('F', 1)],
    'F': []
}

heuristics = {
    'A': 4,
    'B': 2,
    'C': 1,
    'D': 5,
    'E': 1,
    'F': 0
}

def ida_star(start, goal):
    def dfs(node, g, bound, path):
        f = g + heuristics[node]
        if f > bound:
            return f, None
        if node == goal:
            return True, path
        min_bound = float('inf')
        for neighbor, cost in graph[node]:
            if neighbor not in path: # avoid cycles
                result, new_path = dfs(neighbor, g +
cost, bound, path + [neighbor])
                if result is True:
                    return True, new_path
                elif result < min_bound:
                    min_bound = result
        return min_bound, None

    bound = heuristics[start]
    path = [start]
    while True:
        result, found_path = dfs(start, 0, bound,
path)
        if result is True:
            return found_path
        if result == float('inf'):
            return None
        bound = result

# Run IDA*
start, goal = 'A', 'F'
solution = ida_star(start, goal)
print(f"IDA* Path from {start} to {goal}:",
solution)

```

OUTPUT:

IDA* Path from A to F: ['A', 'C', 'F']

Check your Progress 3

1. For the 8-puzzle problem, two heuristics are given: h1: Number of misplaced tiles and h2: Sum of Manhattan distances. Which heuristic will expand fewer nodes and why?

.....
.....
.....

2. In DFS, what happens when the algorithm reaches a node with no unvisited neighbours?

.....
.....
.....

3. An AI agent must find the shortest route in a city map where nodes represent intersections and edges represent roads with distances. Design this as a state space search problem. Explain why A* is preferred over BFS.

.....
.....
.....

11.5 MODEL EVALUATION AND HYPERPARAMETER TUNING IN PYTHON

Once we build a machine learning model, we must evaluate its performance to check if it is accurate and reliable. Common Evaluation Metrics for classification problems are: accuracy, precision, recall, F1-Score, confusion matrix and ROC-AUC Score.

A confusion matrix is a 2×2 (for binary classification) table that summarizes model predictions against actual outcomes. It is represented as follows:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

It helps to visualize where the model is making errors. Using confusion matrix, we can compute accuracy, precision, recall, etc.

Accuracy measures the overall correctness of a model. It tells how many total predictions were correct out of all predictions made.

$$\text{Accuracy} = \{\text{TP} + \text{TN}\} / \{\text{TP} + \text{TN} + \text{FP} + \text{FN}\}$$

Precision measures how many of the predicted positive cases were actually positive. It answers: *“When the model says ‘yes’, how often is it correct?”*

$$\text{Precision} = \{\text{TP}\} / \{\text{TP} + \text{FP}\}$$

Recall measures how many of the actual positive cases were correctly predicted. It answers: *“Out of all the real positives, how many did we find?”*

$$\text{Recall} = \{\text{TP}\} / \{\text{TP} + \text{FN}\}$$

The F1-Score is the harmonic mean of precision and recall. It balances both metrics — useful when you need to find a trade-off between precision and recall.

$$\text{F1-Score} = (2 * \{\text{Precision}\} * \{\text{Recall}\}) / (\{\text{Precision}\} + \{\text{Recall}\})$$

ROC (Receiver Operating Characteristic) curve plots True Positive Rate (Recall) vs False Positive Rate (FPR) at different threshold values. AUC (Area Under the Curve) measures how well the model distinguishes between classes. If AUC = 1.0, it means classifier is a perfect one. And if AUC = 0.5, it means, there is no discrimination (random guessing).

Sklearn provides the function support to find these metrics.

Hyperparameter Tuning - Machine learning models have hyperparameters (settings chosen before training, not learned from data). Examples:

- Number of trees in RandomForest.
- Learning rate in XGBoost/Gradient Boosting.
- Kernel type in SVM.

Choosing the right hyperparameters can dramatically improve performance.

Methods of Hyperparameter Tuning

- Grid Search – Tries all combinations of parameters (refer to 11.3.4 for implementation and syntax).
- Random Search – Tries random combinations (faster).
- Bayesian Optimization / Optuna – Smarter search (advanced).

11.6 SUMMARY

In this unit, we learned about implementation details along with different libraries of Python which are useful for writing AI and ML based code. Further, we have seen Python code for different AI based algorithms such as N-Queen problem, Min-Max algorithm with and without alpha beta pruning, BFS, DFS, Best First search, A*, AO* and IDA* search algorithms. Also, the unit covered the discussion about the evaluation metrics used for classification problems. Finally, the unit discussed the methods of hyperparameter tuning available in python.

11.7 SOLUTIONS/ANSWERS

Check your Progress 1

1.

```
import matplotlib.pyplot as plt

# Sample data
labels = ['AI', 'ML', 'Data Science', 'Cloud', 'IoT']
sizes = [30, 25, 20, 15, 10]

# Draw pie chart
plt.figure()
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
plt.title('Sample Technology Distribution')
plt.show()
```

2.

```
arr = np.array([30, 25, 20, 15, 10])
print(np.sum(arr))
print(np.mean(arr))
print(np.std(arr))
print(np.max(arr))
print(np.min(arr))
```

3. Tensors are multidimensional arrays that can store numerical data — similar to NumPy arrays. They help in deep learning because:

- I. They efficiently represent input data in numerical form.
- II. They support automatic differentiation, which helps in computing gradients during backpropagation.
- III. They can be easily processed on GPUs, enabling faster training of deep learning models.

Check your Progress 2

1. The `is_safe()` function checks if a queen can be safely placed in a specific row and column. It ensures no other queen is already placed in the same column, row, or diagonals, to prevent attacks between queens.
2. When a position is not safe, the algorithm skips that position and tries the next column (or next row). This process continues until a safe position is found or all possibilities are exhausted, leading to backtracking.
3. The base case occurs when the game reaches a terminal state, that is, when the game is over (win, lose, or draw) or when there are no more possible moves. At this point, the algorithm returns the evaluated score of that state instead of continuing recursion.

Check your Progress 3

1. When A^* is applied using given heuristic functions, Manhattan distance heuristic (h_2) will expand fewer nodes because it provides a more accurate estimate of the remaining cost and dominates the misplaced-tile heuristic (h_1).
2. The algorithm backtracks to the previous node to explore other unexplored paths.
3.
State space: Each state represents the current intersection where the AI agent is located.
Initial state: The starting intersection S.
Goal state: The destination intersection G.
Actions / Operators: From a given intersection, the agent can move to any adjacent intersection connected by a road.
Transition model: Applying an action moves the agent from one intersection to another along a road.
Path cost function: The cost of a path is the sum of distances of the roads traversed.
Objective: Find a path from SSS to GGG with minimum total distance.

A^* is preferred over BFS for city route planning because it accounts for varying road distances, guarantees an optimal solution using heuristics, and explores the search space more efficiently.

11.8 FURTHER READINGS

1. S. J. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2021.
2. Géron, Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.
3. Burkov, The Hundred-Page Machine Learning Book, Quebec, Canada: Andriy Burkov, 2019.

UNIT 12: APPLICATIONS OF AI

Structure	Page No.
12.0 Introduction	372
12.1 Objective	373
12.2 AI in healthcare	373
12.3 AI in finance	376
12.4 AI in transportation	379
12.5 AI in customer service and chatbots	382
12.6 AI in education	384
12.7 Summary	386
12.8 Solutions/ Answers	387
12.9 Further Readings	389

12.0 INTRODUCTION

Artificial Intelligence (AI) has evolved into an advanced field of computer science. It significantly transforms multiple domains by enabling machines to think, learn, and act intelligently. With the rapid growth of digital systems, organizations across various domains are generating a huge amount of data. However, traditional computational approaches face challenges in extracting meaningful insights, making timely decisions, and identifying complex patterns from such large-scale data. These limitations highlight the need for intelligent systems that are capable of learning from data and making decisions in dynamic environments.

In healthcare, the increasing complexity of medical data, such as imaging records, electronic health reports, and patient histories, poses significant challenges for accurate diagnosis and timely intervention. Manual processing is time-consuming and may lead to variability, thereby affecting clinical outcomes. Moreover, the growing demand for personalized treatment and early disease prediction places additional pressure on existing healthcare systems.

Similarly, the education sector faces challenges in addressing diverse learning needs, monitoring individual student progress, and ensuring fair and consistent evaluation. Traditional instructional models often struggle to adapt to varying learner capabilities, while educators are burdened with repetitive assessment and administrative tasks, limiting their ability to focus on personalized mentoring.

In finance, increased digital transactions and financial services have exposed financial institutions to risks such as fraud, market volatility, and credit default. Identifying suspicious patterns, managing financial risk, and delivering efficient customer support in real time remain critical concerns for financial institutions operating in highly competitive and regulated environments.

Transportation and industrial systems are also becoming increasingly complex due to urbanization, growing logistics demands, and the need for operational

efficiency. Challenges such as traffic congestion, unpredictable delays, equipment failures, and inefficient resource utilization highlight the limitations of rule-based and manual control mechanisms. Similar concerns arise in manufacturing, where maintaining productivity while minimizing downtime and defects is a persistent issue.

Agriculture faces its own set of challenges, including climate variability, resource constraints, pest management, and the need to improve crop yield sustainably. Traditional farming practices often lack the precision and adaptability required to respond effectively to these evolving conditions.

Beyond these domains, areas such as cybersecurity, environmental monitoring, entertainment, and space exploration are confronted with increasingly complex problem spaces that require systems capable of reasoning under uncertainty and adapting over time. Addressing these challenges necessitates intelligent computational approaches that can learn from experience and support complex decision-making.

12.1 OBJECTIVES

After going through this unit, you will be able to:

1. Understand the importance of Artificial Intelligence (AI) for solving real-world problems
2. Discuss the use of AI in various domains such as healthcare, finance, transportation, customer services and education.

12.2 AI IN HEALTHCARE

Artificial Intelligence (AI) has become one of the important decision-supporting tools in the healthcare sector. With the use of AI, the health sector has become faster, more accurate, and more accessible. AI-based systems can assist doctors in identifying disease stages and personalising the treatment. Unlike traditional methods, AI systems can analyze massive amounts of data in seconds and identify patterns that even highly trained human experts may miss. It can be applied across diagnosis, treatment planning, drug discovery, and medical imaging.

The healthcare system faces lots of challenges worldwide, such as a shortage of doctors, increasing patient demand, and rising healthcare costs. AI acts as a digital assistant for doctors and helps them speed up the work process. This may help in improving patient outcomes and reducing human errors. According to a report by Accenture in 2022, AI applications in healthcare can potentially create \$150 billion in annual savings for the U.S. healthcare system by 2026. Also, NITI Aayog and other initiatives estimate that AI could add hundreds of billions of dollars to India's economy by the mid-2030s, reflecting broad adoption, including in healthcare.

12.2.1 AI in Diagnosis

Diagnosis is one of the most critical stages in healthcare. An early and accurate diagnosis often determines the success of treatment. AI can assist the medical practitioners in diagnosing diseases by analysing electronic health records (EHRs), lab test results, genetic data, and even patient-reported symptoms. Machine learning models can be trained on thousands of past cases to detect signs of diseases such as cancer, diabetes, Alzheimer's, cardiovascular diseases, and rare disorders.

For example, IBM Watson Health uses AI-based systems to analyze patient records and compare symptoms with a vast amount of medical literature to suggest possible conditions. Similarly, AI chatbots such as Babylon Health allow patients to input their symptoms and receive possible diagnoses before visiting a doctor.

Some of the applications of AI-based diagnosis are:

- AI algorithms are widely used in oncology. For example, Google's LYNA (Lymph Node Assistant) can detect metastatic breast cancer with an accuracy of over 92%. Further, AI can also scan through various medical imaging such as mammograms, lung CT scans, and skin images to identify cancerous cells at an early stage.
- AI helps in cardiology by analyzing ECG (Electrocardiogram) and echocardiogram signals to detect heart-related issues.
- AI models are very helpful in Neurology. It can identify early signs of Alzheimer's or Parkinson's by analyzing brain scans and patient behavior data.
- AI can be used to assess an individual's mental health using questionnaires, textual data, facial expressions, voice, etc.

12.2.2 AI in Treatment

AI plays an important role in treatment planning and execution, once diagnosis is done. Some of its applications are as follows:

1. Using AI, precision or personalised treatment can be provided to patients, which means customizing the treatments to individual patients rather than using a "one-size-fits-all" approach. For example, in cancer treatment, AI can analyze a patient's genetic profile to recommend the most effective drug.
2. AI-based robots can act as an assistant to surgeons in performing minimally invasive surgeries with high precision. These systems can analyze real-time surgical data and alert doctors if there's a risk.
3. Developing a new drug is an expensive and time-consuming task. AI helps accelerate the process by predicting how different molecules will interact with diseases. For instance, the company DeepMind

(Alphabet/Google) developed AlphaFold, which can predict protein structures with remarkable accuracy.

4. AI also helps in tracking long-term diseases such as diabetes and hypertension. AI-enabled wearable devices can continuously monitor patient data and generate alerts. For example, Apple Watch and Fitbit use AI algorithms to detect abnormal heart rhythms and warn users.

12.2.3. AI in Medical Imaging

Medical imaging is one of the most advanced and successful applications of AI in healthcare. AI-based computer vision models analyse X-rays, MRIs, CT scans, and ultrasounds to detect abnormalities.

- AI-based systems can assist radiologists by automatically identifying the suspected areas in the scan. For example, AI can help detect the lung nodules in the CT scans that may be an indication of early-stage lung cancer.
- Digital pathology uses AI to analyze the biopsy samples. Instead of manually examining tissue slides, AI models can scan and classify the tissue patterns and give us results faster.
- AI systems like Google's DeepMind AI can detect diabetic retinopathy and macular degeneration from retinal images.
- The AI tools are trained to analyze the skin images to identify melanoma and other skin cancers. Apps such as SkinVision allow patients to upload pictures of skin lesions for AI analysis.

12.2.4. Case Study: IBM Watson for Oncology

IBM Watson for Oncology [1] is one of the most well-known AI applications in the healthcare sector.. It is developed by IBM in collaboration with Memorial Sloan Kettering Cancer Center (MSKCC). This AI system was designed to help doctors in cancer diagnosis and the recommendation of personalized treatment plans. The main goal of the system is to use the vast amount of data available in the medical literature, clinical guidelines, and patient records to help oncologists make fast and precise decisions.

Watson for Oncology uses the latest technologies such as natural language processing (NLP) and machine learning algorithms to read and understand the unstructured medical data such as clinical notes, pathology reports, and research papers. It processes millions of pages of medical information and then compares this knowledge to patient's medical history, test results, and genetic data. Based on the analysis of data, Watson model suggests the potential treatment options which are ranked in three classes as "recommended," "for consideration," or "not recommended," along with the scientific evidence to support each option.

This AI system is in use at several hospitals across many countries, including the United States, India, and South Korea. For example, Manipal Hospitals in India implemented Watson for Oncology to support cancer specialists in

identifying the optimal treatment plans for the patients. Using the system, less time is spent reviewing cases, and more precise treatment can be provided to patients.

Despite its wide use in healthcare, Watson for Oncology faced challenges of adapting to local treatment protocols and rapidly evolving medical data.

12.2.5. Challenges and Ethical Concerns

The ethical concerns and challenges faced by AI-based healthcare systems are:

- Medical data is highly sensitive. It can harm patients, if misused.
- If an AI-based system is trained on non-representative datasets, it may lead to incorrect results, especially in underrepresented populations.
- AI-based healthcare systems can be used to assist doctors. They cannot replace doctors. The final decisions must remain with human experts.
- Who is responsible if an AI system makes a wrong diagnosis?

12.3 AI IN FINANCE

Finance is one of the most data-driven sectors in the world. Every second, banks, stock markets, and insurance companies generate enormous amounts of data. Analyzing this huge amount of data (also called big data) fast and accurately is important for making business decisions. To make timely decisions after due analysis of data, AI based systems can play an important role. They can help to protect us from fraud, and ensure the financial stability.

In the finance sector, AI technologies such as machine learning (ML), deep learning (DL), and natural language processing (NLP) are helping to improve the accuracy, efficiency, and security. It is helping to prevent credit card fraud, executing the stock trades in milliseconds and predicting the risks in the loans and investments.

12.3.1. AI in Fraud Detection

One of the biggest challenges in the financial industry is digital financial fraud. The traditional methods for fraud detection are often based on a fixed set of rules e.g., if a transaction is above a certain amount, generate an alert. However, the fraudsters are constantly evolving their techniques and making the rule-based systems less effective.

AI-based systems use machine learning models to continuously learn from data generated by new transactions and adapt to changing patterns in fraud. These models can detect unusual patterns or anomalies in real time.

Some of the real-world examples in fraud detection are:

1. AI systems can monitor millions of transactions and flag suspicious activity, e.g., if a card is used in two countries within minutes.
2. AI can analyse login attempts and detect abnormal behaviour, such as accessing an account from an unusual device.

3. AI models can scan claims and medical records to detect false claims in insurance.

12.3.2. AI in Algorithmic Trading

Stock trading is a highly competitive and time-sensitive task. Timeliness of the decisions can make a huge difference in outcomes. It can result in significant profits or losses. It is difficult for human traders to analyze market data at very high speed.

Algorithmic trading (or algo-trading) refers to the use of computer programs to execute trades based on pre-defined rules. In these algorithms, AI can be used to learn patterns, predict price movements, and optimize the strategies in real time as follows:

- AI models can analyze historical stock prices, market news, and economic indicators to predict future price movements.
- NLP models can read financial news, tweets, and company reports to identify market sentiment and predict stock fluctuations.
- AI systems can execute thousands of trades per second by rapidly analyzing real-time market data.
- AI algorithms can help investors to balance risk and return by suggesting the best combinations of assets.

Examples of real-world implementation are:

1. JP Morgan's LOXM is an AI-based system which can execute trades with minimal market impact.
2. Goldman Sachs uses AI-based platforms to analyze the investment opportunities.
3. Renaissance Technologies, a hedge fund, relies on AI and ML models for its trading strategies..

12.3.3. AI in Risk Assessment

While granting a loan, insuring a customer, or investing in the stock market, every financial institution should assess the risks. Traditional risk assessment methods are based on credit scores and static rules. AI-based systems can find even the hidden patterns and perform an accurate assessment in different fields such as:

1. AI-based systems can evaluate whether a person is likely to repay a loan by analyzing not just the credit score but also their spending habits, digital footprint, and social behavior.
2. AI-based models can forecast the market volatility and the potential losses by analyzing the real-time financial data, global events, and economic reports.

3. AI-based systems can help banks to detect internal risks like system failures, cyberattacks, or staff misconduct.
4. AI-based models can assess risks in insurance applications. For instance, they can analyze the driving behavior from the telematics devices in the cars to determine fair insurance premiums.

Examples of real-world implementation are:

- ZestFinance uses AI models to analyze the alternative data (e.g., phone bill payments, online behavior) for better credit scoring.
- FICO incorporates AI into its credit scoring system to reduce defaults.
- Swiss Re (insurance company) uses AI to improve underwriting and predict claims risks.

12.3.4. Case Study: JPMorgan Chase

JPMorgan Chase's [2] COiN (Contract Intelligence) platform is one of the most successful and widely mentioned examples of artificial intelligence in the financial sector. The system was developed to automate the process of reviewing the legal and financial documents, which traditionally took thousands of employee hours every year. COiN can analyze the complex commercial loan agreements very fast using ML and NLP. It can accurately identify the important clauses, risks, and compliance requirements.

Manual review of loan documents is a time-consuming process that can be affected by human error as well. Using COiN, JPMorgan has reduced document review time from 360,000 hours to just a few seconds. This system not only saves time and cost but also ensures greater consistency and accuracy in legal and financial operations.

In addition to the contract analysis, JPMorgan Chase also applies AI-based models for fraud detection and transaction monitoring. AI algorithms continuously analyze customer transactions in real-time and identify unusual patterns that may indicate fraudulent activity. These models learn new patterns over time and improve their ability to detect new types of financial fraud that can be missed by traditional rule-based systems. This approach improves customer trust, reduces financial risk, and sets a benchmark for the use of AI in global finance.

Check your Progress 1

1. What are the challenges of using AI in healthcare?

.....

.....

.....

2. How AI can ensure security in the domain of finance?

-
-
-
3. How does bias in medical training datasets affect AI-driven clinical decisions? Suggest possible solutions.
-
-
-

12.4 AI IN TRANSPORTATION

Transportation is one of the most important sectors for society and the economy. Transportation is important as people need to travel for work, education, health care, and leisure. Further, goods need to be transferred from producers to consumers. However, traditional transportation systems impose a number of challenges such as traffic congestion, road accidents, high fuel consumption, and environmental pollution. To cater to these challenges, AI has evolved as a powerful tool. Using machine learning, computer vision, robotics, and big data, AI systems can improve vehicle operations, traffic flows, and transport networks.

Two major applications of AI in transportation are:

1. Autonomous vehicles such as self-driving cars, trucks, and drones.
2. Traffic optimization such as intelligent traffic management, smart routing, and predictive analysis.

12.4.1. Autonomous Vehicles (Self-Driving Cars)

Autonomous vehicles (AVs), also known as self-driving vehicles, are intelligent transportation systems that operate without human intervention by using sensors, artificial intelligence, machine learning, and advanced control systems to perceive their environment, make real-time decisions, and navigate safely with minimal or no driver input under defined operating conditions.

Autonomous vehicles are based on many technologies, such as:

- Cameras, LiDAR (Light Detection and Ranging), radar, and ultrasonic sensors collect information about a vehicle's surroundings.
- AI-based models process sensor data to identify objects such as cars, pedestrians, road signs, and obstacles.
- Vehicles use GPS and AI based algorithms to estimate their position on a map.
- Machine learning models help to decide actions like braking, accelerating, or changing lanes.

- The computer system of the car can map the decisions into movement through steering, throttle, and brakes.

The Society of Automotive Engineers (SAE) defines a six-level stack of driving automation, i.e. from Level 0 (no automation) to Level 5 (full automation without human input). Most current commercial systems like Tesla Autopilot or GM Cruise are developed around Level 2 or 3, which indicates that they require human intervention at some point in time.

Some of the real-world examples of the autonomous vehicles are:

- Google's self-driving project, Waymo, operates the autonomous taxis in cities like Phoenix, Arizona.
- Tesla's Autopilot uses computer vision and neural networks for semi-autonomous driving.
- Nuro focuses on autonomous delivery robots for groceries and small goods.
- Uber ATG previously developed self-driving taxis (sold to Aurora in 2020).

Some of the challenges of the autonomous vehicles are as follows:

- The safety and reliability concerns if an autonomous car gets involved in some accident.
- The LiDAR sensors and other high-performance AI based systems are very expensive.
- The roads, signals, and the communication networks need upgradation for full automation.
- Many people still hesitate to trust AI with their safety.

12.4.2. AI in Traffic Optimization

AI can be used to improve the existing transport systems by making them smarter. Cities around the world often struggle with traffic congestion, long commute times, increased fuel consumption, air pollution, greenhouse gas emissions and hence, economic losses. To overcome these problems, AI can be used for predicting traffic flow, optimizing signal timings, and guiding vehicles through the best routes.

Some of the applications for traffic optimization are:

- AI-based smart traffic lights can adjust signal timings based on real-time traffic flow. Instead of fixed timers, the signals can adapt it dynamically.
- Machine learning-based predictive analysis can help in predicting the traffic jams in advance by analyzing the historical data, weather and accidents.

- The GPS based smart navigation systems such as Google Maps and Waze use AI techniques to suggest the fastest routes.
- Using connected vehicles technology, the cars can communicate with each other which can help in cooperative driving.
- AI can help in bus scheduling, demand forecasting, and route planning to reduce waiting time.

Some of the real-world applications of AI-based traffic optimization are:

- Surtrac (Pittsburgh, USA) is an AI-based traffic light system that reduces travel time by 25% and wait time by 40%.
- Google Maps can predict traffic delays and suggest some alternative routes using machine learning.
- Singapore Smart Nation uses sensors and analytics for integrated traffic monitoring.
- China's Smart Cities uses AI for traffic signals optimization and emergency response.

12.4.3. Case Study: Tesla AI-Based Autonomous Vehicles

Tesla's self-driving car [3] is one of the widely known examples of artificial intelligence in transportation. AI-based projects such as Autopilot and Full Self-Driving (FSD) systems make the vehicles smarter so that they can navigate, steer, accelerate, and apply brakes automatically under various conditions. These systems are created using deep learning, computer vision, and sensor fusion, which allows the car to sense the surrounding and make decisions in real time.

Tesla vehicles are equipped with multiple cameras, ultrasonic sensors, and radar systems that can continuously capture environmental data. This data is processed using neural networks to identify the objects on the road, such as traffic lights, pedestrians, lane markings, other vehicles, and road signs. Unlike the traditional rule-based systems that depend on a fixed set of instructions, Tesla's autonomous vehicles learn from the massive amount of real-world driving data collected from millions of cars worldwide, which makes the model smarter.

The core idea behind Tesla's approach is the state-space search algorithms and decision-making, where the constant evaluation of all possible actions (such as turning, stopping, or accelerating) takes place to select the safest and most efficient option. This process happens within a short time and ensures the smooth and safe navigation even in complex traffic situations.

Tesla's AI-driven transportation technology has revolutionized the automotive industry, ranging from autonomous driving, traffic optimization to road safety improvements. It also supports the energy efficiency by optimizing the routes and the driving patterns. While fully autonomous driving is still under

development and faces regulatory challenges, Tesla's autonomous vehicles represent a major step toward the future of smart mobility, where vehicles can communicate, learn, and make decisions without human intervention.

12.5 AI in Customer Service and Chatbots

Across most industries, customers need support from service providers to answer their questions and listen to their complaints. Good customer service increases customer trust and satisfaction. Earlier, customer service was provided by human customer care executives who used to provide support on phone calls, emails, or in-person interactions. With the evolution of AI, a significant transformation has taken place in the customer service industry. One of the most significant innovations in this area is the use of chatbots and AI-based virtual assistants as alternatives to human service executives. These technologies allow companies to provide 24/7 customer support, reduce costs, and improve customer satisfaction.

12.5.1. AI based techniques to improve Customer Service

The following AI-based techniques are used to improve customer service:

1. Natural Language Processing (NLP) is used by chatbots to understand and communicate in human language.
2. Machine Learning can be used to analyse customer feedback and recommend more effective responses for future interactions.
3. Sentiment analysis can be used to detect the emotions of the customer i.e. whether a customer is angry, confused, or satisfied, and adjust responses accordingly.
4. AI can make personalized recommendation of products or services based on customer browsing history and preferences. This makes interactions more useful and relevant.

12.5.2. Applications of AI in Customer Service

Various applications of AI-based systems in Customer Service are:

- Unlike human agents, AI-based chatbots are available 24/7. They can handle customer requests around the clock, across different time zones.
- During peak hours, AI-based chatbots can manage thousands of queries simultaneously.
- Many banks, e-commerce platforms, and applications use AI-based chatbots to guide new customers for initial setup and registration process.
- Retailers use these chatbots to help customers to check delivery status, request refunds, or change orders.
- AI-based customer service may provide troubleshooting steps for the common problems before escalating it to a human executive.

- AI-based chatbots can provide support in multiple languages.

12.5.3. Case Study: Amazon Alexa

Alexa [4] is a voice-based AI assistant developed by Amazon. It is one of the most widely used AI-powered virtual assistants for customer support. It improves the customer interaction, generates automated responses, and provides a personalized assistance. It uses various latest technologies such as NLP, ML and speech recognition to understand spoken commands, process user intent, and respond naturally.

It allows users to perform a number of tasks such as checking product prices, placing orders, tracking deliveries, and managing subscriptions. All these tasks can be done simply using voice commands. Technically, Alexa uses rule-based state-space search to identify user intent. It then matches the identified intent with a suitable response and executes the required actions. Over time, its machine learning algorithms continuously learn from millions of interactions to improve accuracy and provide more personalized assistance.

By using Alexa as a virtual customer care executive, organizations can reduce their dependence on human call-centre agents. Some companies are integrating this virtual customer care executive with their customer service to provide 24/7 support. For instance, many smart home device manufacturers now use Alexa to help customers set up and control their products using conversational instructions. The advantages of using virtual customer care executive include faster response time, 24X7 availability, improved user satisfaction and less overheads.

Check your Progress 2

1. How do sensor fusion techniques (LiDAR, radar, cameras) improve decision-making in autonomous vehicles, and what limitations still exist?
.....
.....
.....
2. What is the role of sentiment analysis in the case of AI based customer services?
.....
.....
.....
3. Discuss the potential risks of relying heavily on AI for customer complaint resolution.

.....

.....

.....

12.6 AI IN EDUCATION

Education serves as the foundation of human progress. Traditional teaching methods often follow a one-size-fits-all approach, where all students are expected to learn the same content at the same pace. However, learners differ significantly in their abilities, preferences, and learning speeds. Some students grasp concepts more quickly than others, while some prefer visual explanations over textual content. Others require additional practice questions rather than advanced, challenging material.

AI has transformed the education system by enabling personalized learning experiences for individual students. AI-based Intelligent Tutoring Systems (ITS) function as virtual tutors, providing customized instruction tailored to each learner's needs. These technologies enhance learning effectiveness and also assist teachers in classroom management and monitoring student progress.

12.6.1. Personalized Learning with AI

- Personalized learning involves adapting teaching content, speed and learning style according to a student's needs. Instead of teaching everyone in the same way, AI system can adjust according to how well a student is performing. For example, if one student struggles with algebra, the AI system can provide additional exercises, step-by-step hints or video explanations on the specific topic to the student so that s/he can practice more on that topic. If another student finds the same topic easy, the system can give more advanced problems from the same topic to keep them challenged.
- Some of the features of AI-based personalized learning systems are:
- Khan Academy and DreamBox are using AI-based tools to adjust the difficulty of questions according to the student's needs in real time.
- These tools also present the learning analytics (test scores, speed, mistakes) of the student to find the patterns and identify the weak areas.
- These AI-based tools also recommend the lessons, exercises, or videos either based on a student's weak areas or in an incremental approach.
- AI-based learning games motivate the students by rewarding progress with points, badges, or levels.
- Some of the real-world examples of personalized learning are:
- Knewton (Pearson) adapts digital textbooks according to the individual student's needs.

- Coursera AI Recommendations can suggest courses to students based on their progress and interests.
- BYJU's (India) used AI for adaptive video lessons and quizzes.

12.6.2. Intelligent Tutoring Systems (ITS)

- An Intelligent Tutoring System (ITS) is a computer program that provides one-to-one tutoring to learners, similar to that offered by a human teacher. It uses AI to understand the student's response, provides hints, corrects mistakes, and offers feedback in real time.
- The components of ITS are:
- The domain model stores knowledge about the subject (e.g., math rules, grammar).
- The student model tracks the student's learning pace, weaknesses, and misconceptions.
- The tutoring model decides which lesson or hint should be provided to the student next.
- The user interface allows the student to interact with the system using text, voice, or visuals.

Some real-world examples of ITS are

- Carnegie Learning (Cognitive Tutor) is an AI-based tutor for mathematics.
- AutoTutor helps the students in science and technology subjects using natural language dialogue.
- Duolingo uses AI-based tutoring for personalized language learning.

12.6.3. Case Study: Duolingo

Duolingo [5] is a very well-known language learning platform that allows users to learn new languages. It uses AI to provide a personalized and adaptive learning experience to millions of users worldwide. It combines rule-based AI and machine learning to make learning engaging, effective, and customized according to each learner's ability.

It analyzes the pattern of users' interaction with given lessons and tasks to record the response time, accuracy, and mistakes. These recorded parameters are further used to predict the learner's proficiency level. Based on this data, the app dynamically adjusts the difficulty level of the exercises and the pace of new lessons. This approach is based on the state-space search concept where AI continuously evaluates the learner's current "state" and determines the optimal next "state" or lesson to maximize progress.

A rule-based system ensures that the grammar and the vocabulary rules are presented in some logical sequence. The machine learning algorithms improve the experience of personalization by identifying the weak topics for focused practice. Duolingo also uses NLP to evaluate pronunciation and the written answers to provide instant feedback similar to a human tutor.

The gamification feature of the app, such as streak, points, and leaderboard, is also based on AI algorithms. It maintains the user motivation by giving adaptive rewards. By combining psychology, linguistics, and artificial intelligence, Duolingo makes learning fun and scientifically effective.

This case study highlights the transformation in the field of education from a one-size-fits-all model to a personalized, interactive, and continuous learning experience using AI-based techniques. It shows that AI can act as a virtual tutor that is capable of understanding the learners' needs, adapting to their pace, and helping them to achieve better outcomes.

Check your Progress 3

1. Explain how AI-based adaptive learning systems improve student performance.

.....

.....

.....

2. How might over-reliance on AI tools affect students' critical thinking and problem-solving skills?

.....

.....

.....

3. How should teachers' roles evolve in classrooms increasingly supported by AI technologies?

.....

.....

.....

12.7 SUMMARY

This unit explores various applications and growing importance of Artificial Intelligence (AI) across a number of domains such as healthcare, finance, transportation, customer services and education. AI has become a transforming

force by enabling machines to process huge volume of data. It can learn patterns and experience from available data. It can perform complex tasks with high speed and improved accuracy.

In healthcare, AI provides support in medical imaging, disease prediction, drug development and personalized treatments with improved diagnosis accuracy. In financial sector, AI can help in improving security by fraud detection, automated trading, credit score estimation and improved customer services using intelligent chatbot.

Further, transportation system also has been revolutionized by AI based self-driving vehicles, intelligent traffic management and route optimizations which contributes to a safe and efficient mobility. In customer services, AI based chatbot and virtual assistant provides instant support round the clock which improves user satisfaction and reduces the operation cost. Similarly, in education AI provides adaptive learning system with personalized feedback and automated grading. It ensures that the teaching and learning process become more effective and student centric.

Across all these domains, in addition to enhanced efficiency, AI based models also provide advanced analytical capabilities and data-driven insights for effective decision making. However, it also brings some challenges such as data privacy concerns, ethical issues and need of human oversight. Overall, AI has become indispensable technology which is reshaping the industries and redefining the human interaction with machines. The unit emphasizes that the understanding of AI based applications and its implications is essential for preparing students and professionals to participate effectively in increasingly intelligent and automated world.

12.8 SOLUTIONS/ ANSWERS

Check your Progress 1

1. The challenges of using AI in healthcare include data privacy issues, lack of quality medical data, high implementation costs, and the need for expert supervision.
2. AI ensures security in finance by detecting fraud, monitoring transactions for unusual patterns, and protecting customer data using advanced algorithms.
3. Bias in medical training datasets can lead to inaccurate diagnoses and unequal treatment recommendations, particularly for underrepresented populations. This may worsen healthcare disparities and reduce trust in AI systems. Solutions include using diverse and representative datasets, continuous bias auditing, transparent algorithms, inclusive clinical validation, and involving multidisciplinary experts to ensure fairness and reliability.

Check your Progress 2

1. Sensor fusion integrates data from LiDAR, radar, and cameras to create an accurate and reliable perception of the environment in autonomous vehicles. It enhances object detection, distance measurement, and decision-making by combining complementary strengths and ensuring redundancy. However, limitations include high computational demands, sensor calibration issues, weather sensitivity, cost constraints, and difficulties handling rare or unpredictable road scenarios.
2. Sentiment analysis helps AI-based customer services to understand the emotions of the customer and the feedback which allows them to respond more accurately and improve user satisfaction.
3. Relying heavily on AI for customer complaint resolution may reduce human empathy and personalization, leading to customer dissatisfaction. AI systems may misinterpret complex queries, sarcasm, or emotional tone, resulting in inappropriate responses. Data privacy concerns, algorithmic bias, system errors, and over-automation can further damage trust, especially when critical or sensitive issues require human judgment and discretion.

Check your Progress 3

1. AI-based adaptive learning systems improve student performance by delivering personalized instruction tailored to each learner's needs, abilities, and pace. These systems continuously analyze student performance data to identify strengths, weaknesses, and knowledge gaps. Based on this analysis, they adjust the difficulty level of content, recommend targeted practice, and provide immediate feedback. This real-time support helps students correct mistakes quickly and reinforces conceptual understanding. Additionally, adaptive systems promote self-paced learning, reducing frustration and improving confidence. By enhancing engagement through interactive and data-driven approaches, AI-based adaptive learning systems increase knowledge retention, academic achievement, and overall learning effectiveness compared to traditional methods.
2. Over-reliance on AI tools may weaken students' critical thinking and problem-solving skills by reducing active cognitive engagement. When learners depend heavily on AI-generated answers, they may focus more on obtaining solutions than understanding underlying concepts. This can limit analytical reasoning, creativity, and independent judgment. Additionally, constant assistance may discourage perseverance in tackling complex problems. Students might also struggle to evaluate the accuracy or bias of AI outputs, affecting their ability to think critically.
3. In AI-supported classrooms, teachers should evolve into facilitators, mentors, and ethical guides. Rather than merely delivering content, they

must interpret AI-generated insights, foster critical thinking, and ensure responsible technology use. Teachers should use learning analytics to personalize instruction while providing emotional support, creativity, and human interaction that AI cannot replace, ensuring balanced and meaningful learning experiences.

12.9 FURTHER READINGS

- S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson Education, 2021.
- E. Rich, K. Knight, and S. B. Nair, *Artificial Intelligence*, 3rd ed. New Delhi, India: McGraw Hill Education, 2017.
- S. Chopra, *Artificial Intelligence and Machine Learning: Fundamentals and Applications*. New Delhi, India: Khanna Publishing, 2022.
- S. Jain, K. Pandey, P. Jain, and K. P. Seng, *Artificial Intelligence, Machine Learning, and Mental Health in Pandemics: A Computational Approach*, 1st ed. Academic Press, 2022. ISBN 9780323911962.
- P. J. Rousseeuw, *Artificial Intelligence and Big Data in Financial Analytics*. Cham, Switzerland: Springer, 2022.
- R. Baker and K. Yacef, *The State of Educational Data Mining in 2020: AI in Learning and Teaching*. New York, NY, USA: Springer, 2020.

12.11 REFERENCES

- [1] Jie, Zhou, Zeng Zhiying, and Li Li. "A meta-analysis of Watson for Oncology in clinical application." *Scientific reports* 11.1 (2021): 5792.
- [2] <https://d3.harvard.edu/platform-digit/submission/robo-banking-artificial-intelligence-at-jpmorgan-chase/>
- [3] <https://www.tesla.com/AI>
- [4] <https://www.britannica.com/technology/Amazon-Alexa>
- [5] <https://www.duolingo.com/>